CubeCat

A Comprehensive Online Hub For Speed cubers



Final Year Project report for a degree in MComp Computer Science at Sussex University

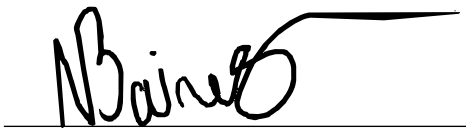
Candidate 279103
Supervised by Dr. Alessio Santamaria

UNIVERSITY OF SUSSEX

Academic year 2025/26

Statement of Originality

This report is submitted as part requirement for the degree of Computer Science at the University of Sussex. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged. I hereby give permission for a copy of this report to be loaned out to students in future years.



Signed — Candidate 279103

Summary

This project began as an attempt to create a singular resource with everything an aspiring speed cuber would need to develop and learn new skills. This *online hub* would be called *CubeCat*, a name derived from the naming scheme I use for all of the major software projects I have written, as is the logo displayed on the title page of this report, the pixel art style cat is the main logo I use in my projects.

Over the course of development however, I encountered significant issues in the implementation of the main algorithm which would be required for this task, the solving algorithm known as *Thistlethwaite's 52 move algorithm*. The sparsity and complexity of the documentation of this algorithm made it very difficult to understand and implement.

As a result the goal of this project changed from a comprehensive hub for speed cubers, to an investigation and documentation of my implementation of Thistlethwaite's algorithm. While this is not the original goal I had hoped to achieve, I am happy with what I achieved during this project. Programming, Mathematics, and Speed cubing are subjects which I have always felt a deep personal connection to, and despite the challenges I faced, I am very glad for the opportunity to have connected them in this report.

In the end I managed to create a program which generates the four lookup tables required by Thistlethwaite's algorithm, and a web page which allows the user to 'paint' a 3x3x3 Rubik's Cube, ensure the cube they have painted is valid, and then displays the solution with each turn animated, and descriptions on each stage of the algorithm used to solve the cube.

Contents

1 Introduction **5**
- 1.1 Project summary . . . 5
- 1.2 Report structure . . . 5
- 1.3 Aims and Motivation . . . 5
- 1.4 Professional Considerations . . . 6
  - 1.4.1 Public Interest . . . 6
  - 1.4.2 Professional Competence and Integrity . . . 6
  - 1.4.3 Duty to Relevant Authority . . . 6
  - 1.4.4 Duty to the Profession . . . 7

2 Theoretical Introduction **8**
- 2.1 Theory Outline . . . 8
- 2.2 The 3x3x3 Rubik's Cube . . . 8
- 2.3 Mathematical framework of the Rubik's Cube . . . 9
  - 2.3.1 The Cube Group . . . 9
  - 2.3.2 Solving the cube . . . 9
- 2.4 Cube State Validation . . . 17
  - 2.4.1 What makes a cube state valid? . . . 17

3 Development **20**
- 3.1 Problem Specification . . . 20
- 3.2 Requirements Analysis . . . 20
  - 3.2.1 Analysis of Solver Specification . . . 20
  - 3.2.2 Analysis of Timer Specification . . . 21
  - 3.2.3 Analysis of Algorithm Database . . . 22
- 3.3 Implementation . . . 22
  - 3.3.1 Solver . . . 22
  - 3.3.2 Timer and Speedcubing Database . . . 32
- 3.4 Evaluation . . . 32

4 Conclusion **34**
- 4.1 Success of the Implementation . . . 34
- 4.2 The Bliss of Hindsight . . . 34
- 4.3 Future Extensions . . . 34

A Interim Log **38**
- A.1 Meeting 1 — 2nd October 2025, 4pm . . . 38
- A.2 Meeting 2 — 16th October 2025, 4pm . . . 38
- A.3 Meeting 3 — 30th October 2025, 4pm . . . 39
- A.4 Meeting 4 — 6th November 2025, 4pm . . . 39
- A.5 Meeting 5 — 4th December 2025, 4pm . . . 39
- A.6 Meeting 6 — 3rd February 2026, 10:30am . . . 40
- A.7 Meeting 7 — 17th February 2026, 10:30am . . . 40
- A.8 Meeting 8 — 17th March 2026, 10:30am . . . 40
- A.9 Meeting 9 — 21st April 2026, 10:30am . . . 40

B Project Proposal **41**
B.1 Aims and Objectives 41
B.1.1 Aims 41
B.1.2 Objectives 41
B.2 Relevance 43
B.3 Resources Required 43
B.4 Weekly timetable 43

C UI Design **44**

Chapter 1

Introduction

1.1 Project summary

In this project, I set out to create a website which provided training resources for new speed cubers. Being a speed cuber myself, I felt the sparsity, separation, and relative lack of a common standard posed a hindrance to people trying to learn how to solve a Rubik's cube, and improve their times / skills.

As I will demonstrate over the course of this report, such a comprehensive online hub was, perhaps too ambitious given my time frame, and the difficulty / lack of resources surrounding the key elements of my project. The core algorithmic problem is an implementation of Thistlethwaite's 52 move algorithm [Sch15b], and while some resources are available about it, I find a lot of those resources either not applicable to my specific situation, using some bizarre variation, or nigh on incomprehensible. Because of this, the implementation of this algorithm proved much more challenging and time consuming than I had originally anticipated, as a result, I ended up deciding not to go forward with implementing specifically the community algorithm database. I decided to remove this part of my project since, after receiving feedback on my interim report, I felt that the implementation of that part of my original design to be not as critical to my success, where the implementation of the solver application and supporting algorithms to be more necessary.

1.2 Report structure

This report is structured into four main sections:

- Introduction
- Theoretical Introduction
- Development
- Conclusion

Each section is, I feel, relatively self-explanatory as to its' contents. Notably the Theoretical Introduction section covers key elements of the theory that I did not cover sufficiently in my interim report.

1.3 Aims and Motivation

I have been a Speed cuber for many years. While I was learning, I made use of many different resources scattered across the internet to improve my skills or learn new techniques. The aim behind this project was to attempt to consolidate these resources into a single page, and to implement a cloud storage solution to storing solve session statistics.

I was diagnosed as Autistic in 2012, and while it has not affected me to such extreme degrees as others, I have tended to find certain things in life more difficult than neurotypical people. There

have been certain activities throughout my life that have formed escapes from these difficulties, such activities that... *quiet* the world. Programming is one such activity, two others include Speed cubing, and Mathematics.

Something I have always wanted to do is to combine these things, but have always felt limited to. This project has created an opportunity to research into fields which interact with all three of these interests, and despite the difficulties it has posed, I have thoroughly cenjoyed it.

1.4 Professional Considerations

1.4.1 Public Interest

Due to the lack of user testing required by this application, its' development will not involve any parties outside of myself and my supervisor, Dr. Santamaria.

This application likely has little impact on public health / well being, but the implementation of a user account system requires that I ensure the privacy and security of user information. The legitimate rights of third parties shall be protected through mechanism which allow access to user data, thus ensuring that the data protection rights of potential users are met.

Any user management and associated application systems will not discriminate against users / people in any way based on their identity or other attributes.

Any user data which is required and stored by the application will be done in a safe and secure manner, meeting the current industry specifications for data protection.

Making use of existing optimised cloud hosting platforms could help to improve environmental considerations. This project in general however, should have little environmental impact.

1.4.2 Professional Competence and Integrity

This project is within my level of professional competence, and documentation / standards relevant to this project that I must make use of to complete this project will be used to develop my own knowledge of my field. I will admit and attempt to rectify any lack of competency I have in areas respective of this project. I have been a programmer and speed cuber for many years, and a lot of the technical / academic skills required by this project have been covered over the course of my undergraduate degree, and personal studies in the years prior and during.

I will ensure I understand and follow any relevant legislation throughout the development of this project. I will avoid malicious intent or negligent action / inaction, and injury to others. To my knowledge there is no particular legislation or risk to health of anyone involved in this project, besides that which is relevant to data protection. Legislation relevant to data protection will be conformed to.

I will respect the alternative viewpoints of others, and consider them in the development of my own understanding of the field, and where possible will offer / seek / accept honest criticisms of my own work, and the work of others.

Finally, I will reject and not make any offer of bribery or unethical inducement.

1.4.3 Duty to Relevant Authority

I shall carry out my responsibilities with the due care and diligence required, and seek to avoid any situation which may give rise to any conflict of interest.

I accept full responsibility for my work, and future responsibility for any colleague who may be working under my supervision. Furthermore, I will not disclose any confidential information provided to me during the course of my work, unless legislation or my authority requires me to do so.

I will not misrepresent or withhold information on the performance anything I produce, unless I am lawfully bound to do so. I will not take advantage of the lack of relevant knowledge of others.

1.4.4 Duty to the Profession

This project does not damage the reputation of my profession, in the same way that existing similar projects have little impact that on the reputation of Computer Science and its' related fields.

By following the rules laid out by The Chartered Institute for IT, I am upholding their reputation and standards, and I fully intend to act with integrity and respect of others I may encounter in a professional setting.

As I mentioned previously, this project in particular should not involve any parties outside of myself and Dr. Santamaria, whom I respect for his work and efforts in teaching.

Chapter 2

Theoretical Introduction

2.1 Theory Outline

The main theoretical part of this application is the implementation of a Computational solving algorithm for the 3x3x3 Rubik's Cube.

There are a few algorithms which can perform this task, the one I will implement here is Thistlethwaite's 52 Move algorithm, which we will look at in detail in a later section of this part of the report.

Before we can discuss this algorithm however, we must establish the mathematical framework used in Rubik's Cube theory, and the structure of the cube itself.

2.2 The 3x3x3 Rubik's Cube

The 3x3x3 Rubik's Cube was a popular rotating puzzle created by Ernő Rubik in 1974 [Edi25], and gained significant popularity during the 1980s.

The cube is comprised of 26 smaller cubes, referred as Cubies, which are connected through an internal structure which allows each face of the cube to rotate.

Each of the 26 cubies is constructed of a number of facelets, the individual coloured squares. The number of facelets and the position of the cubie on the cube define whether the cubie is an edge piece, a corner piece, or a centre piece.

The cube contains 8 corner cubies, 12 edge cubies, and 6 centre cubies, the corners having 3 natural orientations in any position, the edges having 2 orientations, and the centres having 4, an orientation is the physical twist of a cubie relative to its home position and twist, I will discuss in more detail how orientation is defined for each type of cubie later on.

We need not pay much attention to the position / orientation of the centre cubies, since they are fixed in position relative to each other and the internal structure of the cube, and their orientation is not easily observable on the cube since they consist of only one facelet, and as such would always appear in the same orientation.

Given this information, Richard E. Korf calculates the number of permutations which can be achieved through the 3x3x3 Rubik's Cube as $43,252,003,274,489,856,000$ [Kor97].

Turns of each face of the cube are commonly denoted using Singmaster's Notation [Joy08]:

- U — 90° clockwise turn of the upper face.
- F — 90° clockwise turn of the front face.
- L — 90° clockwise turn of the left face.

$$\frac{8! \times 3^8 \times 12! \times 2^{12}}{12} = 43,252,003,274,489,856,000 \tag{2.1}$$

Figure 2.1: Permutations of the 3x3x3 Rubik's Cube

- B — 90° clockwise turn of the back face.
- R — 90° clockwise turn of the right face.
- D — 90° clockwise turn of the bottom face.

There are two main modifiers which can be applied to each move, if a move is of the form x', it denotes a 90° anticlockwise turn of the x face, and the form x^2 denotes a 180° degree turn of the x face.

Each slice of between faces of the cube (containing only centre and edge cubies) is also assigned a name:

- The M (middle) slice — The slice between the R and L faces.
- The E (equatorial) slice — The slice between the U and D faces.
- The S (standing) slice — The slice between the F and B faces.

Using this notation we may also refer to individual cubies. A cubie is denoted as a combination of the letters above, depending on which faces the cubie is adjacent to. For example the corner cubie which is common to the top, front, and right faces of the cube would be the UFR cubie. Similarly the edge cubie common to the left and bottom face would be the LD cubie.

Throughout the development of this project I will use a set of indexes to reference cubies, and positions on the cube (See Figure 2.2). Other indexes will be based on the same order presented in this index. For example, in places I may use the corner index, which contains only the corner cubies indexed on $[0, 8[$, and the edges indexed on $[0, 12[$.

2.3 Mathematical framework of the Rubik's Cube

The Rubik's Cube is approached with an application of group theory, as many puzzles commonly are.

2.3.1 The Cube Group

The Cube Group G, is the group containing all possible permutations of the Rubik's Cube. The elements of G are achieved from combinations of the previously mentioned turns of each face, U, F, L, B, R, and D [Dav06].

We know the number of elements this group contains (see Figure 2.1).

2.3.2 Solving the cube

Clearly such a problem space is far too large to solve in a reasonable amount of time, to get around such an issue, solving algorithms commonly use an approach of decomposition, reducing the overall problem to a series of less complex sub problems, each with a much smaller problem space.

The solving algorithm we will use is called Thistlethwaite's 52 move algorithm, one of the first computational approaches to solving the cube which leveraged the behaviour of the cube under the paradigm of group theory [Sch15b][1].

Thistlethwaite's Algorithm breaks the problem down into 4 stages. Each stage is solved using a pregenerated lookup table which maps a cube state to a sequence of moves which will take said cube to a state in which the sub problem is solved. In order to create said lookup tables, we must devise a way to encode the cube which captures only the elements of the cube that are solved by the current stage, then, starting from a solved cube, we can perform a breadth first search until we have found all of the possible cube states going into the stage.

Each stage prevents works done in the previous stage from being undone by enforcing an increasingly restrictive set of moves that are allowed when searching for cube states.

I will refer to the lookup table that takes the cube from stage n to m as $G_{n\to m}$.

[1] There are very few good references I could find for this algorithm. While this citation is more or less a blog site, it contains what is reportedly a typed version of a letter written by Thistlethwaite which describes this algorithm. I struggled to find any official publication of Thistlethwaites algorithm beyond this.

1. UBL
2. UB
3. UBR
4. UL
5. U
6. UR
7. UFL
8. UF
9. UFR
10. LB
11. B
12. RB
13. L
14. R
15. FL
16. F
17. FR
18. DBL
19. BD
20. DBR
21. LD
22. D
23. RD
24. FDL
25. FD
26. FDR

Figure 2.2: The overall cubie index

Thistlethwaite's Algorithm — Stage One

The first stage of Thistlethwaite's Algorithm is concerned with the orientation of the edge cubies. The orientation of an edge as defined by Thistlethwaite is not necessarily intuitive, the idea is that when the U and D^2 face are turned clockwise 90°, we regard the edges on the face which was turned as having been flipped. We call an unflipped edge 'good', and a flipped edge 'bad' [Sch15b]. This stage is solved when all edges are good, the following set of moves are allowed in this stage: $\{U, D, F, B, R, L\}$ [3].

Given that this stage is only concerned with the orientation of the edges, the cube encoding for this stage which will be used for this stages lookup table need only contain information about the orientation of the 12 edges. Using this knowledge, we can calculate the number of states we would expect to find in this stage's lookup table (see Figure 2.3).

$$|G_{0\to1}| = 2^{11} = 2048 \tag{2.2}$$

Figure 2.3: The number of permutations expected in $G_{0\to1}$. Note that the exponent is 11 and not 12 as one might expect. This is because the orientation of the twelfth edge is determined by lookup at the preceding 11 edges [Sch15b].

Thistlethwaite's Algorithm — Stage Two

For the second stage, our move set is restricted to $\{U^2, D^2, F, B, R, L\}$. This prevents the edge orientations (which were solved in the previous stage) from changing, since our definition of edge orientation means that edges are flipped b quarter turns of the U and D faces, by restricting only half (180°) turns of these faces, the edges cannot be flipped.

This stage aims to solve the corner orientations, and place the edges which belong in M slice in the M slice. Note that the edges do not need to be solved, but must at least be in this slice by the end of this stage.

We must then, define what we mean by 'corner orientation'. Every corner has a facelet which belongs on either the R or L face, and every corner will also always be adjacent to one of these faces. If the corner's R/L facelet is on the R/L face, then the corner is in its natural orientation, if not, then the corner's twist is determined by the twist that must have been performed to get the R/L facelet to its current position. See Figure 2.4 for examples of this.

[2] Note that you may choose any pair of opposite faces, but your choice of faces must remain consistent throughout the stage

[3] This is effectively any move which was previously discussed (except for turns of the slice layers), because one may combine the moves in this set to achieve any modified form of face turn. For example $RRR = R'$.

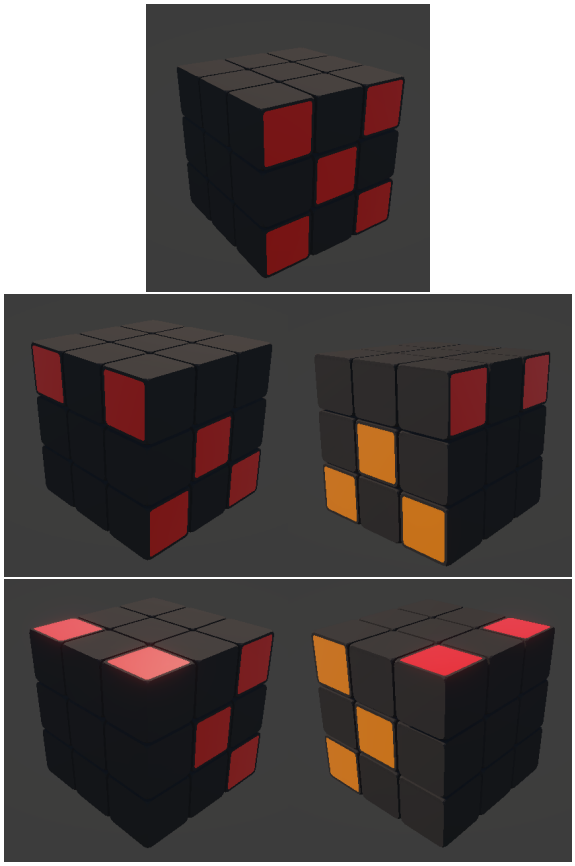


Figure 2.4: An example of how corner orientation is defined. We have a cube on which only the L / R facelets of the corners are painted, and the centre cubies. In the first image, each corner is in its natural, or 'home' orientation. In the second image, we can see that the red facelets of the top corners have been twisted clockwise away from the adjacent L / R face, and in the third image, we can see that the left / right corners have been twisted anti-clockwise away from their adjacent L / R face.

The encoding for this stage then, must contain the current positions of the edges which belong in the M slice, and the current orientation of the corners.

Since the edges do not need to be solved into their home positions, and only into their home slice, for every permutation of the M slice edges, there are $4! - 1$ other permutations of those edges which can be solved in the same way. In other words, the M slice edges will be arranged among four positions on the cube, it does not matter which specific edge is in which position, all that matters are those four positions. Hence, under the goal of this stage, every 4! permutations of the M slice edges can be solved with the same permutation. This must be incorporated into our encoding, in my implementation I do this by getting the positions of the four M slice edges, sorting them by an index I created to enumerate cubie positions, and inserting the sorted positions into the encoding, this meant that the 4! positions which should be solved in the same way resulted in the same encoding, and thus were not inserted into the lookup table. See Figure 2.5 for an example of this.

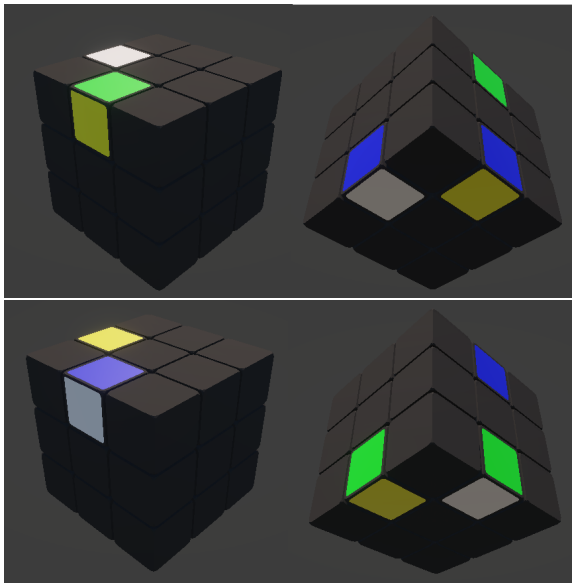


Figure 2.5: Here we have two examples of cases that would be solved in the same way. All four of the painted edges must be solved into the same slice. Note that in each image, the left and right views are from opposite corners of the cube. If we were solving the edges into their home positions, i.e. the positions they would take on the solved cube, these two cases would have different solutions, but since we are just solving them into their home slice, they have the same solution. They might not end up in their home position, but the same solution will take all four of the edges into their home slice in these cases.

See Figure 2.6 for the number of permutations in this stage.

$$|G_{1\to 2}| = 3^7 \times \binom{12}{4} = 1082565 \tag{2.3}$$

Figure 2.6: The number of permutations in table $G_{1\to 2}$ [Sch15b].

Thistlethwaite's Algorithm — Stage Three

In this stage we are allowed the following set of moves, $\{U^2, D^2, F^2, B^2, R, L\}$.

The third stage of this algorithm is the most theoretically complex. The specification I have been following states that the purpose of this stage is to:

1. Solve the E and S slice edges into their home slices (like we did for the M slice in the previous stage).
2. Solve the corner's into their *tetrads*.
3. Ensure each corner tetrad has a total twist of zero.
4. Ensure the parity of the corner permutation is even (ensuring this will also ensure that the edge permutation is even) [Sch15b].

A Tetrad is a three dimensional shape consisting of four triangular faces, and four vertices. One can imagine each set of four corners on the cube forming a Tetrad, by considering each of the four corners as vertices on the Tetrad. See Figure 2.7 for an example of the two Tetrads forming a cube.

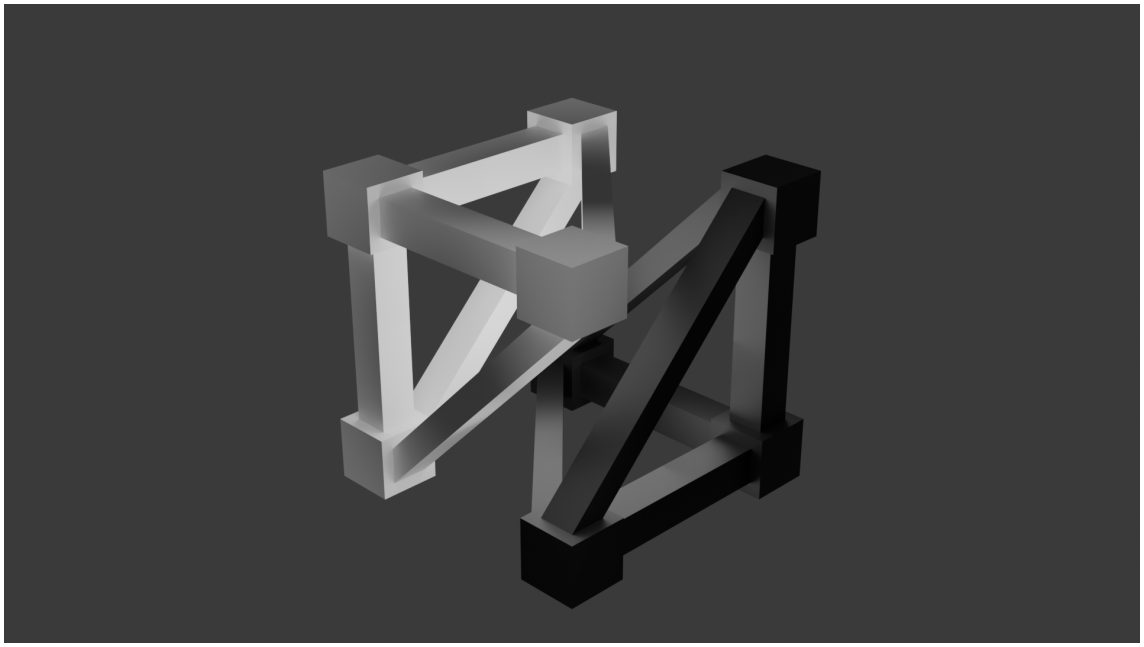


Figure 2.7: An example of the Tetrad arrangement in a cube.

A permutation is an arrangement of elements of a set. A common way of discussing permutations is using Disjoint cycle notation [Gro08]. This notation represents a permutation as a composition of independent n cycles. An n cycle, is of the form $(a_1\ a_2\ \dots\ a_n)$, it describes a cycle of n elements from the set, when performing the permutation, each element would take on the position of the next element mentioned in the cycle, in this example, a_1 would take the position of a_2, a_2 would move to a_3, and so on. Upon reaching the last element, one wraps to the start of the cycle, so a_n would move to a_1. An n cycle can be decomposed into a composition of independent transpositions, or two-cycles. There are many general formulas for determining the decomposition of an n cycle, because the specific result found by one formula is not unique, one might decompose a permutation in many valid ways. I will be using the one found in Figure 2.8. The decomposed form of an n cycle can be used to determine its parity. The parity of any n cycle is the same as the parity of the number of transpositions in its decomposition, i.e. if a cycle decomposes to 2 transpositions, it is even, if it decomposed to three transpositions, it would be odd [NIS]. An example of an F turn of the cube written in disjoint cycle notation can be found in Figure 2.9. Another notation I will use a lot is the binomial coefficient notation, this and the related Permutation formula, are very important for calculating a lot of the figures I will / have used in this report. The binomial coefficient can be used to calculate the number of unique subsets of size r, which can be formed of a set of size n, while the permutation formula calculates the number of non-unique subsets which can be formed of a set, non-unique in this context meaning subsets which contain the same elements, but in a different order. See Figure 2.10 for the formal defintions of this formulae.

$$(a_1\ a_2\ \dots\ a_n) = (a_1\ a_2)(a_2\ a_3)\dots(a_{n-1}\ a_n) \tag{2.4}$$

Figure 2.8: General formula for the decomposition of an n cycle into independent transpositions.

$$(UFL\ UFR\ DFR\ DFL)(UF\ RF\ DF\ LF) \tag{2.5}$$

Figure 2.9: A single F turn represented in disjoint cycle notation, where the permuted elements are the cubies themselves. Each set of brackets is an independent cyclic permutation, i.e. cubie UFL moves to the UFR position, the UFR cubie moves to the DFR position, the DFR cubie moves to the DFL position, and the DFL cubie moves to the UFL position, etc...

$$\begin{aligned} \binom{n}{r} &=^n C_r = \frac{n!}{r!(n-r)!} \\ P(n,r) &=^n P_r = \frac{n!}{(n-r)!} \end{aligned} \tag{2.6}$$

Figure 2.10: The Binomial coefficient [Goe87] and Permutation formula [Cha02], for a set of size n, and subset size r.

So far this is fine, however I couldn't find a nice definition of *Tetrad twist* anywhere. In my search I did find a forum post[4] by Jaap Scherphuis which attempted to describe this tetrad twist. He visualised it as 'You can think of each tetrad having a U/D axis, going through the centre of the face containing its U-pair of corners through to the opposite face. The two tetrads need to have their U/D axes aligned', While this seems simple enough in concept, in practice it turned out to be quite complex to implement, and following my research on this topic there appears to be very very few implementation of this algorithm which actually take this approach for stage 3, they mostly seem to use variations which are not quite as efficient as Thistlethwaite's definition, which are more simply defined, but reach further into stage 4.

During said research, I came across a variation by Stefan Pochmann[5], wherein he arranges the corners into pairs, and solves them into these pairs, rather than solving into their tetrads and then solving their tetrad twists. He states that 'in phase 3 my subgoal is more restrictive', he doesn't (as far as I can tell), provide any formal proof as to whether it is actually more restrictive, and does not provide any proof as to the number of permutations for the lookup table in his stage, so I will have to determine this myself.

The other parts of this stage are as in Thistlethwaite's algorithm, we have factors of $\binom{8}{4}$ and 2 for solving the edges into their slices, and making the corner permutation even [Sch15b], but the factors for solving the pairs are slightly trickier.

For the first pair, we have eight corners to choose from, and two positions they must be arranged between, which implies $\binom{8}{2}$. For the second pair, we have six corners, and another two positions, so we have $\binom{6}{2}$. For the third pair we have four corners, and another two positions, so $\binom{4}{2}$. Following this pattern the final pair would have $\binom{2}{2} = 1$, so we can ignore this factor in our calculations.

[4]StackExchange — Is it possible to calculate group 3's factor of 3 in Thistlethwaite's Algorithm
[5]Stefan Pochmann — Thistlethwaite Solver

Finally then, we have our factor for solving the corner pairs,

$$\begin{aligned} \binom{8}{2} \times \binom{6}{2} \times \binom{4}{2} &= P \\ {}^nC_r = \frac{n!}{r!(n-r)!} \therefore P &= \frac{8!}{2!6!} \times \frac{6!}{2!4!} \times \frac{4!}{2!2!} \\ &= \frac{8!6!4!}{(2!)^4 6!4!} = \frac{8!}{(2!)^4} \\ 2! = 2 \therefore P &= \frac{8!}{2^4} = \frac{8!}{16} = 2520 \end{aligned} \tag{2.7}$$

See Figure 2.11 for the final calculation of the size of this stage.

$$|G_{2\to3}| = \frac{8!}{16} \times \binom{8}{4} \times 2 = 352800 \tag{2.8}$$

Figure 2.11: The number of permutations in $G_{2\to3}$

The encoding for this stage then should include the current locations of each of the eight edges to be solved, with both sets of four edges sorted into ascending order to ensure that the edges are solved only into their home slice, not necessarily their home positions as we did in the previous stage. We must also include the index of the pair each corner is currently in, this approach will ensure that the corners are not solved to a position, but simply to their pair. Finally, we must include the parity of the corner permutation.

Thistlethwaite's Algorithm — Stage Four

In this stage we are allowed only half turns, $\{U^2, D^2, F^2, B^2, R^2, L^2\}$.

This is probably the simplest stage yet, we must simply encode the current location of every cubie. Note that we can reduce their locations to indexes of each type of cubie, rather than the set of cubies as a whole, i.e. rather than indexing every cubie $[0, 26[$ [6], we can index the corners on $[0, 8[$, and the edges on $[0 - 12[$, just to reduce the number of bits required for this encoding. If we did not use this different index, every cubie would need to be encoded with five bits, for a total of 20 cubies, this would require 100 bits. Using the different indices, corners can be encoded with three bits, and edges with four, so we have $8 \times 3 + 12 \times 4 = 72$ bits

This stage ultimately solves the cube. See Figure 2.12 for the number of permutations in this stage.

$$|G_{3\to4}| = \frac{4!^5}{12} = 663552 \tag{2.9}$$

Figure 2.12: The number of permutations in $G_{3\to4}$ [Sch15b].

Generating Thistlethwaite's tables

The lookup tables required for this algorithm will be generated using a Breadth First Search [BW] through cube states. We will start with a solved cube, and then perform all valid moves on the cube at each depth, recording the move sequences required to reach each unique encoding. The search should complete when the search queue has emptied, which should occur when the predicted number of permutations for that table has been reached.

Of course, searching through *every* cube state is a thoroughly poor idea, since as we discussed before, there are over 43 quintillion permutations of the cube (See Figure 2.1). To mitigate this, there are a number of optimisations we can use to prune branches of the search tree as we go.

[6] The range $[a, b[$ is the set of integers satisfying $a \leq x < b$.

Firstly, it is obvious that we can prune branches which begin with the inverse of the last move performed, for example if our current move sequence is $U\ F'\ R\ L$, we can ignore the succeeding branch beginning with L', ($U\ F'\ R\ L\ L'$), because such a move sequence is equivalent to $U\ F'\ R$.

Secondly, turns of opposite faces are independent, for example the sequences $F\ B$ and $B\ F$ would result in identical permutations. Given this, we can assign an order in which opposite faces must be turned, and prune any branches which break this order. For example, we might say that an L move can follow an R move, but an R move cannot follow an L move [Kor97].

Finally, if we encounter an encoding which is *not* unique, and has already been recorded in the table, we may discard its entire subtree. This is because if the encoding has already been found, its sub-tree has already been searched (or is in the search queue).

With the first and second optimisations, we reduce our search's branching factor from a maximum of 18 (six face turns, each with three variations), to an asymptotic branching factor of roughly 13 [Kor97].

Another optimisation that could be made is multithreading. Concurrent execution of a Breadth First Search is, in my opinion, a potentially interesting problem, since there are, I think, lots of approaches that might be taken. I will take what I think is the simpler approach here, which is to have multiple threads operating on the same search queue, with the same set of valid moves. This approach would of course mean that a thread safe implementation of the search queue, and table structure would be required. I would implement such structures using Mutexes, such an approach would mean locking a structure's mutex when a thread is accessing it, and unlocking the mutex when it has finished. This approach would result in an amount of idle time for each thread, but ensures that the search is executed successfully, and does not introduce any race conditions or resource locks.

Solving the cube using Thistlethwaite's algorithm

The lookup tables for each stage provide a set of key-value pairs, the key is the stage specific encoding of the cube, and the value is the permutation which was performed to the solved cube in order to reach that encoding. Thus in order to solve the cube using these tables, one must lookup the cube's current state in $G_{0\to1}$, reverse the resulting move sequence, and then perform that move sequence on the cube. One must then repeat this with $G_{1\to2}$, and so on, until the permutation yielded by the final table ($G_{3\to4}$) is performed, resulting in a solved cube.

Notably, the lookup tables will contain the identity permutation for each stage, that is, if the cube is aready in what the current stage would consider to be a solved state, the permutation yielded by the stage's lookup table will contain no moves (hence *identity permutation*).

2.4 Cube State Validation

One of my main goals for this application is to allow users to 'paint' in a cube state, which is then solved. The users would paint the cube one facelet at a time, this means that they would be able to create cube states which cannot be achieved, or are simply not valid. Hence we must discuss a cube state validation algorithm, which will look at the cube the user is currently painting, and determine problems or inconsistencies in its state, and feedback this information to the user.

2.4.1 What makes a cube state valid?

As I see it, there are five properties of the cube that we must observe:

- The number of occurrences of each colour of facelet.
- The configuration of the centre facelets.
- Whether each cubie is unique and contains a valid combination of colours.
- Is the total orientation of the edge and corner pieces valid?
- Is the overall permutation parity even?

In the following sections I will discuss each of these in more detail.

The number of occurences of each colour of facelet

There must be nine of each coloured facelet on the cube.

If this condition is met, we can proceed to the next check, if not, the cube cannot be in a valid state.

The configuration of the centre facelets

On the 3x3 Rubik's Cube, the cubie's rotate around a fixed internal structure. This internal structure forms a cross with the six centres of each face, thus every cube rotates around the centre of a face, but the centre pieces can never move relative to each other.

On any real world cube (with the standard configuration of colours), you will find that the white centre is always opposite the yellow centre, the blue centre is always opposite the green centre, and the red centre is always opposite the orange centre.

Furthermore, if you hold the cube such that the white centre is on the top, and the green centre is on the front, then the orange centre will always be on the left.

Once again, if this condition is not met, the cube cannot be in a valid state.

Whether each cubie is unique and contains a valid combination of colours

Every cubie on the Rubik's cube contains a unique combination of coloured cubies. That combination must also be valid.

For example, there must be a corner cubie with the colours green, white, and orange, but there cannot be an edge piece with the colours red and orange.

Is the total orientation of the edge and corner pieces valid?

Consider a quarter turn of a face. There are four edges on a face, each with two facelets. Four of those facelets will be permuted in one cycle, and the other four will be permuted in another cycle. This is an even permutation. The parity of permutations when combined with other permutations follows the same rules as parity of numbers through addition and subtraction, thus we can only ever achieve even permutations of edges, since the most basic operation we can perform on the cube, which forms every other opertion, follows an even permutation. A flipped edge represents an odd permutation, since it is a transposition of two facelets, thus we can only have even numbers of flipped edges on a valid cube. [NN]

By the same logic we can determine a similar case for corners. Four corners are permuted through a quarter turn of a face, each corner has three facelets, thus 12 facelets are permuted in three independent cycles. Thus we can only achieve states where the corner orientations are a multiple of three (supposing we number each orientation of a corner as 0, 1, and 2).

Therefore, a valid cube must have its total edge orientation be even, and its total corner orientation be a multiple of three.

Is the overall permutation parity even?

A quarter turn of a face is effectively two cycles of length four, one cycle of the edges, and one of the corners. Each of these four cycles can be shown to be odd by decomposing it into a composition of transpositions (See Figure 2.13). Thus the turn of the entire face, is a composition of two odd permutations, which yields an even permutation. [NN].

$$(a\ b\ c\ d) = (a\ b)(b\ c)(c\ d) \tag{2.10}$$

Figure 2.13: A decomposition of a 4-cycle permutation into independent transpositions using Figure 2.8, proving that any 4-cycle has an odd parity, since we yield three independent transpositions.

Therefore, we can only ever achieve an even overall parity of the cubie's permutation. Any cube state with an overall odd cubie permutation cannot be valid.

How does this guarantee a valid cube?

Together, it can be shown that these five properties guarantee a valid, and therefore solvable, cube state.

The first three points are simply observations about the construction of the cube, these criteria must be fulfilled for the cube painted by the user to even resemble a Rubik's Cube in the first place.

The last two points are derived from the fact that any other state is not achievable from the operations we can perform on the cube. As we have previously discussed, we have a total of six basic operations we can perform on the cube, each of those operations can be generalised to a single permutation on the cubies on a face of the cube, and in the preceeding sections we have already seen that these operations can only produce cube states where the overall parity of the cubies is even, the total corner orientation is a multiple of three, and the total orientation of the edges is even.

Therefore, since these are the only operations valid on the cube, these five criteria are sufficient to guarantee a valid cube state.

Chapter 3

Development

3.1 Problem Specification

In this project I am aiming to create three applications.

1. Solver specification 3x3x3 Rubik's Cube solver
   (a) Implementation of Thistlethwaite's Algorithm & Lookup table generator.
   (b) Server which accepts cube permutations and returns a sequence of moves to solve said permutation.
   (c) Solver site
      i. Allow users to paint a cube to the state they wish to solve
      ii. Site determines permutation and sends to the server to solve
      iii. Site animates the solution step by step
      iv. Information about the algorithm used to solve the cube and each stage should be displayed.
2. Timer specification Solve timer
   (a) Scramble generator. Should conform to the WCA specification [WCA25a].
   (b) Timer interface
   (c) Session & Solve statistics
   (d) Account system
   (e) Cloud storage of sessions & session statistics.
3. Algorithm DB specification Algorithm database
   (a) Listing of common solving cases
   (b) Allow users to upload move sequences for common cases
   (c) Overall view of session statistics
   (d) Upvoting / reporting system for submitted move sequences
   (e) Admin interface for reviewing reports & site statistics

3.2 Requirements Analysis

3.2.1 Analysis of Solver Specification

This part of the application takes the form of a website which allows the user to 'paint' a cube state, which it then validates and solves.

Thistlethwaite's algorithm makes use of some potentially large lookup tables, if these were included in this web page it would mean that any client which wants to use the site would need to download the tables upon request, which will slow down response times. I have dealt with this by implementing a seperate 'solving server' application, which loads the solution tables and waits to receive cube permutations to solve, and returns the solution obtained from the tables.

I have implemented a separate program to generate the lookup tables required by Thistlethwaite's algorithm. The solving site and table generator program make use of different cube models to represent the state of the cube, which I will cover later in this report.

The user is able to paint a cube state onto an empty cube using the provided UI (See the low-fi prototype Figure C.1). The site also provides real-time feedback on the validation state of the cube, the validation algorithm identifies areas on the cube which are preventing it from being a valid cube, and relays this information to the user so that they might fix the state they have entered, thus ensuring that the cube they enter is solvable.

Once the site obtains the solution from the server, it displays this solution at the top of the page stage by stage, with information about the current stage of the solution. This structure helps to explain what is being done during the solve, and serves as a basis for this being a teaching tool to learn different solving algorithms.

I implemented the solving site as a Unity WebGL application, and the table generator / solving server in C++. I have also implemented the solving server such that it is able to serve the solving site as well as an endpoint for producing solutions.

3.2.2 Analysis of Timer Specification

The timer provides an interface for timing a solve. This is controlled in a similar way to a *Speed stacking timer* (See Figure 3.1), which is a timer commonly used in speed cubing competitions. The solve is started by the user holding down the spacebar for a short time, once they release the spacebar, the timer will start. Upon solving the cube, the user should then touch any key on the keyboard to stop the timer and record the solve. See Figure C.2, and Figure C.3 for a low-fi UI design of this page.



Figure 3.1: A speedstacking timer

The timer also provides scrambles, this scrambler conforms to the World Cubing Association (WCA) specification, 'An official scramble sequence must produce a random state from all states that require at least 2 moves to solve (equal probability for each state).' [WCA25a]. The number of moves required to solve the cube is verified using our implementation of Thistlethwaite's algorithm.

The timer also records each solve performed in a session and record performance metrics, these metrics being:

- Average of Five (Ao5)
- Mean of Three (Mo3)
- Best single solve
- Overall average

Note that Ao5 is calculated by taking the previous five solves, removing the fastest and slowest, and taking the mean average of the remaining three solves [WCA25b].

These solving sessions are then stored in a persistent database, linked to a user account system, which allows users to make use of different devices for their speedcubing practice.

This part of the project is implemeneted as a JavaScript application hosted by a simple static file host server, which can be implemented as part of the server mentioned in the last section.

3.2.3 Analysis of Algorithm Database

This part of the application provides a place for users to share solutions to common cases in popular speedcubing algorithms.

Common human solving methods are be presented, once a method is selected, a list of common cases is be presented, the user is then be able to submit move sequences which solve each case for other users to see. Other users are then be able to upvote move sequences they like, or report ones they don't like / don't solve the case they are intended to solve. This is all be managed through a backend database system like MySQL.

Users are also able to see an overall view of their solving session statistics from the timer page here.

The site also provides an admin interface, which can only be accessed by user accounts marked as admins. This admin site provides various management features, for example:

- The ability to review reports of move sequences.
- The ability to block / restrict user accounts on the site.
- View overall statistics about the site, such as daily connections, number of users, number of cases submitted, etc...

I have provided a number of low-fi UI prototype designs for this part of the application:

- Figure C.4
- Figure C.9
- Figure C.5
- Figure C.6
- Figure C.7
- Figure C.8

3.3 Implementation

3.3.1 Solver

Generating Thistlethwaite's solution tables

This is the first part of the project I implemented, and also the one that proved the most difficult.

My original plan was to implement a cube model in Java, and use that model to generate each of the tables. This approach yielded success for the first table. After just a few days I had a working cube model, and a correctly generated $G_{0\to1}$.

$G_{1\to2}$ proved *much* more challenging to implement, and I ran into a number of issues along the way. There were a number of small inconsistencies in my cube model, mainly in the way I

was keeping track of cubie orientations. The edge orientations were not difficult to track, See Figure 3.2 for the pseudocode of my approach. I was originally tracking corner orientation in the same way, except bounded to three values instead of two, and only updating when the R or L face was twisted. After I had been struggling with the second stage for a while I eventually looked closer at this approach and realised it was wrong. Tracking corner orientation was a much more complex task than this, since it is dependent on both the corner's location on the cube, and its' current orientation. After exploring how corner orientation behaves on my own cube, I determined the following rules, these rules only make sense if you keep in mind that Thistlethwaite's definition of corner orientation is entirely relative to the R / L faces.

- Any turn of the R / L face will not affect a corner's orientation.
- Turns of the $\{U, D, F, B\}$ faces result in a cyclical orientation of the corners on the turned face.
- Corners joined by the diagonals of a face will experience the same change in rotation.

What I mean by the second point more specifically is that, if you look at a single corner as it rotates around a face, you will notice that its orientation changes on the first turn, reverses on the second turn, changes again on third turn to the same orientation as after the first turn, and then reverses again on the fourth turn, in what is effectively a two-cycle.

Figure 3.2 Tracking edge orientation

```
function EdgeTwist(edge, faceTwisted, clockwise)
    if faceTwisted = U or faceTwisted = D then
        if clockwise then
            edge_orientation ← (edge_orientation + 1)  mod 2
        else
            edge_orientation ← (edge_orientation − 1)  mod 2
        end if
    end if
end function
```

But the orientation of a corner after a turn is nonetheless dependent on its location and orientation at the start of the turn. I solved this problem by determining the two-cycles of orientations of each face turn, illustrated in Figure 3.3. The idea in my approach is that depending on the face which was turned (i.e. what face the corner we are updating is on), we match the corner's *original* position on the face to its *original* orientation, then increment this index in the direction of the turn (+1 for clockwise, −1 for anti-clockwise) to find the orientation the corner should be in after the turn.

Figure 3.3 Tracking corner orientation

function CornerTwist(corner, faceTwisted, clockwise)
  if $faceTwisted = R$ or $faceTwisted = L$ **then return**
  else if $faceTwisted = U$ **then**
    $orientationCycles \leftarrow \{\{0,2,0,2\},\{1,0,1,0\},\{2,1,2,1\}\}$
    $positions \leftarrow \{0,2,8,6\}$
  else if $faceTwisted = D$ **then**
    $orientationCycles \leftarrow \{\{0,1,0,1\},\{1,2,1,2\},\{2,0,2,0\}\}$
    $positions \leftarrow \{17,19,25,23\}$
  else if $faceTwisted = F$ **then**
    $orientationCycles \leftarrow \{\{0,2,0,2\},\{1,0,1,0\},\{2,1,2,1\}\}$
    $positions \leftarrow \{6,8,25,23\}$
  else if $faceTwisted = B$ **then**
    $orientationCycles \leftarrow \{\{0,2,0,2\},\{1,0,1,0\},\{2,1,2,1\}\}$
    $positions \leftarrow \{2,0,17,19\}$
  end if
  for i in $[0,4[$ **do**
    if $cornerPositions_i = corner_{location}$ **then**
      $cornerIndex \leftarrow i$
    end if
  end for
  $cornerIndex \leftarrow \left(cornerIndex + \begin{cases} -1 & \text{if } clockwise \\ 1 & \text{otherwise} \end{cases}\right) \mod 4$
  for i in $[0,4[$ **do**
    $nextTwist \leftarrow \left(cornerIndex + \begin{cases} 1 & \text{if } clockwise \\ -1 & \text{otherwise} \end{cases}\right) \mod 4$
    $corner_{orientation} \leftarrow orientationCycles_{i_{nextTwist}}$
  end for
end function

While this approach meant that I was technically calculating the table correct, my program was just too slow to feasibly calculate the entire table, since $G_{1\to2}$ is the largest of the four tables. Even with the optimisations discussed in section 2.3.2. The problem was in the nature of my implmentation, up to this point I had implemented the program in Java. Java is the language I have the most experience in, and this is why I chose to use it for this project. A big feature of the language is its' garbage collection, this is the process which cleans up memory references which are no longer needed by the program. This of course is a good thing, because it means less work needs to be done by the programmer, since they do not need to clean up their own memory usage. The way Java has garbage collection implemented however, presented the problem.

While a portion of the garbage collection routine is processed asynchronously, there is a part which must block the main program thread to complete. For normal amounts of heap memory this is fine, and this process does not block for long, but in my attempts to get this program working, I manually increased the heap space a lot. This meant that the blocking part of the garbage collection was taking multiple seconds to complete, which meant that as the program went on, it took longer and longer to execute. It was clear to me that my Java approach was flawed in this way, and that I would need to take a new approach in a language which did not have such issues.

My choice of language was C++, this language enforces manual memory management, which meant that it would have no garbage collection routine blocking the program's execution, although I would have to ensure that memory was manually freed.

After re-implementing my cube model and the first table in C++, I attempted $G_{1\to2}$ again, and was successful. The program was in fact significantly faster than my Java implementation. I also implemented the multithreading optimisation discussed at the end of section 2.3.2. I chose to use eight threads in my implementation, since my laptop (the device I used to develop this project) has 16 processor cores, so I used eight threads with the logic that half the processor cores could be dedicated to generating the tables.

The third table was similarly difficult, because I was using a variant which was not particularly well documented. I started by trying to determine the number of expected entries into the table for this variant of the stage, and as documented in Figure 2.11, I was successful. Determining the encoding which would solve the corners into their pairs, and the edges into their slices, was simple enough, the difficult part was determining the permutation parity of the corners. After some reasearch into the subject, I found Pochmann's implemenentation of his variation of this stage, his method of calculating corner permutation parity is illustrated in Figure 3.4.

Figure 3.4 Pochmann's Corner parity [Poc]

```
function CornerParity(cube)
    parity ← 0
    for i ∈ [0, 8[ do
        j ← i + 1
        while j ∈ [0, 8[ do
            a ←GetCorner(i)
            b ←GetCorner(j)
            parity ← parity ⊕ (a_home > b_home)
        end while
    end for
     return parity
end function
```

Essentially, for every corner, we iterate over every succeeding corner (given that corners are indexed on $[0, 8[$), and perform an exclusive or on the current parity value, with a boolean value which is true if the home position of the ith corner is greater on our index than the home position of the jth corner. The idea here is that if this boolean value is true for the ith and j^{th} corner, these corners *must* have been swapped, and thus form a transposition. The exclusive or is then determining whether the number of occurrences of this condition is even. If the result is zero, the parity is even, if the result is one, the parity is odd.

This approach correctly determined the corner parity, and solved $G_{2\to3}$.

I had no issues with the implementation of the final table $G_{3\to4}$, it was implemented as discussed in my previous section discussing it.

Solving server

The solving server was implemented as expected. I used a HTTP library written for C++ to implement a custom HTTP server which accepts a cube permutation in a 'multipart/form-data' body, calls upon the solution tables to solve the permutation, and returns the solution move sequence in a simple 'text/plain' body.

The cube permutation accepted by the solver should be determined with the pseudocode found in Figure 3.5. Note that the encoding of the cubies is derived from the knowledge that my cubie index (See Figure 2.2) contains 26 values and thus can be contained within five bits, and the orientation of a cubie has at most four values, and thus can be contained by four bits.

Figure 3.5 Algorithm for determining a cube permutation to be accepted by the solving server.

```
function PERMUTATION(cube)
    buffer ← 0
    i ← 0
    for cubie in cube do
        cubieEncoding ←CUBIEENCODING(cubie)
        buffer ← buffer | LEFTSHIFT(cubieEncoding, i × 8)
        i ← i + 1
    end for
     return buffer
end function
function CUBIEENCODING(cubie)
    p ← cubie_position
    o ←LEFTSHIFT(cubie_orientation, 5)
    if cubie is center cubie then
        o ← 0
    end if
     return o|p
end function
```

Solver page

The solver page implementation itself didn't pose too many challenges. The main difficulty was implementing the program in such a way that it was independent of the colours on the cube, a problem which I will discuss in the next section on my validation algorithm.

The solver page shows an unpainted 3x3 cube, the user then uses the coloured buttons and the bottom of the page to colour in this cube to represent the one they want to solve. Once they hit the 'Solve' button, they are presented with a new UI, which shows the solution steps at the top of the screen, along with three buttons, one to step forward a move, one to step backward a move, and one to play the entire solve automatically.

The page determines the specific permutation of the cube from the user's painted facelets, and sends this to the solving server to determine the solution, this separation means that Thistlethwaite's tables (which can be quite large), do not have to be downloaded to the client.

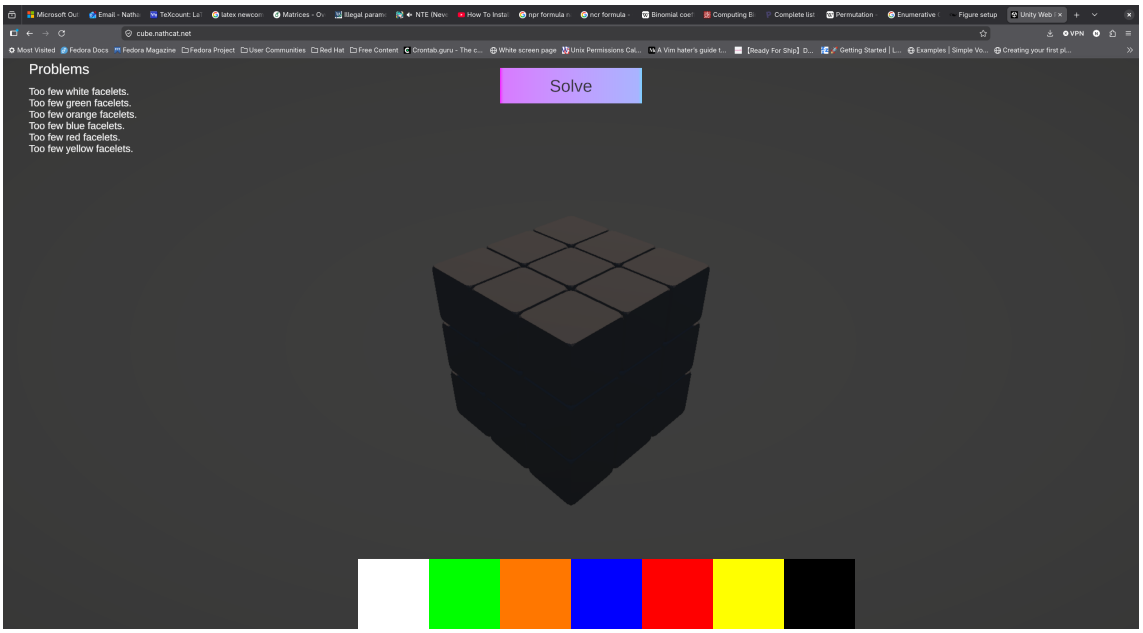


Figure 3.6: The solving page, ready for the user to paint their cube.

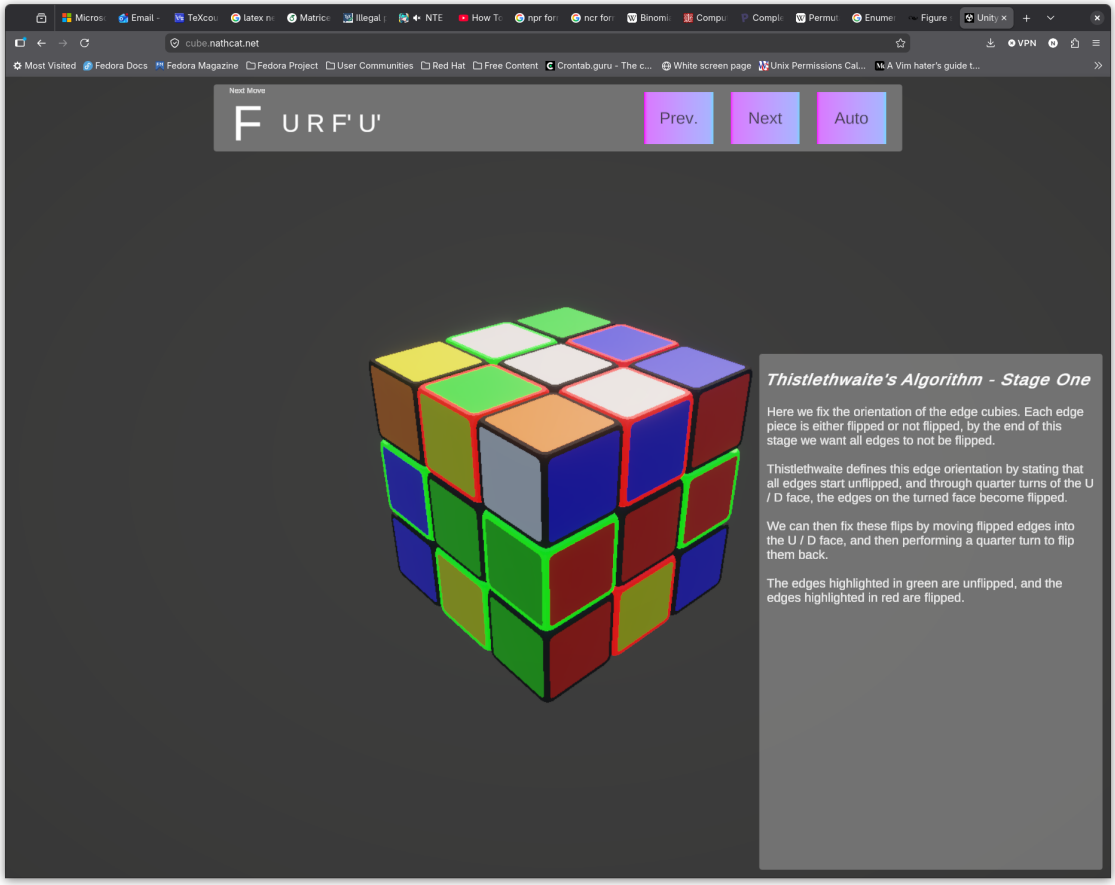


Figure 3.7: Solving the cube, in stage one of Thistlethwaite's algorithm.

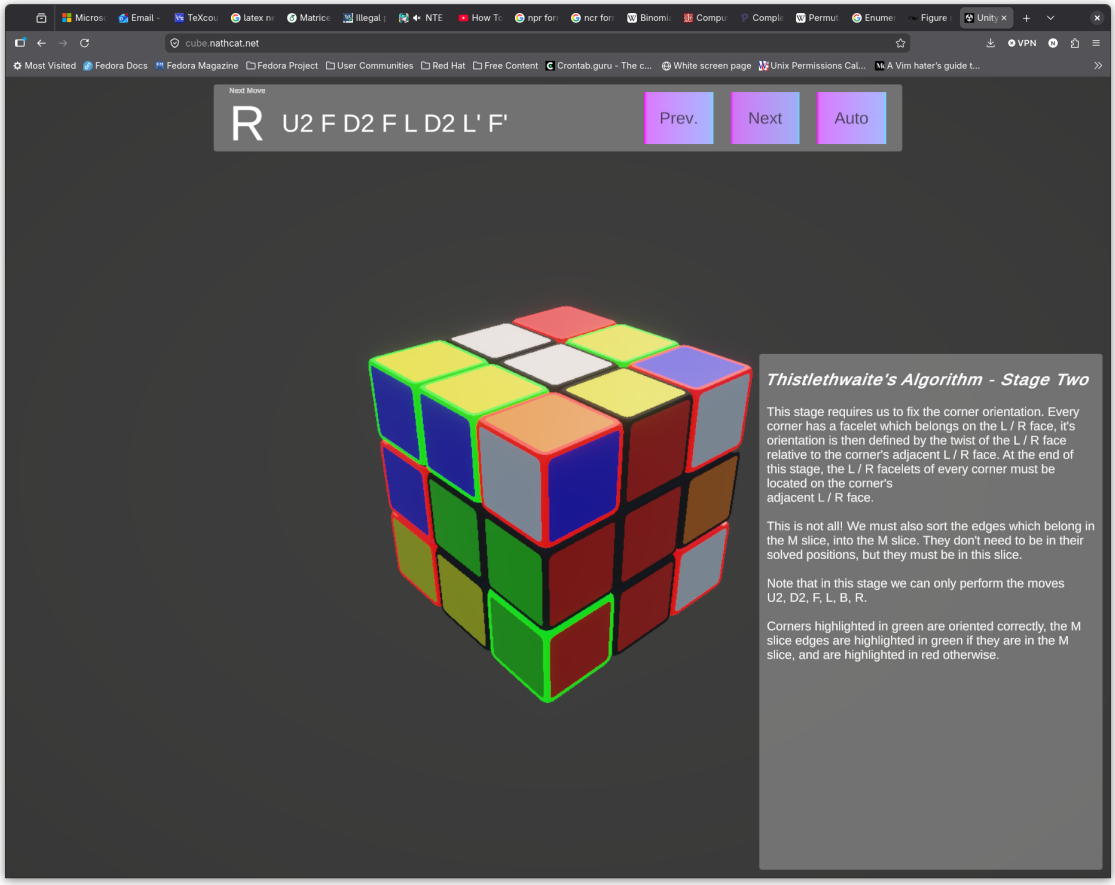


Figure 3.8: Solving the cube, in stage two of Thistlethwaite's algorithm.

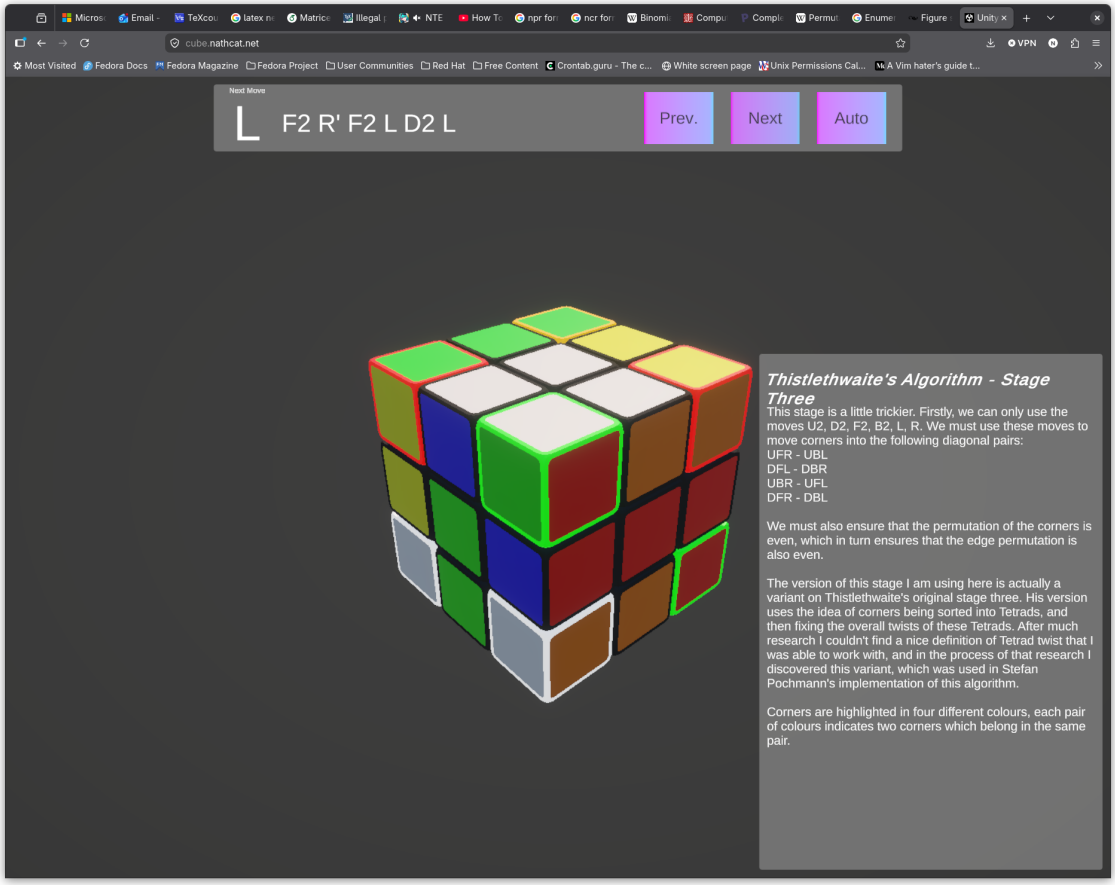


Figure 3.9: Solving the cube, in stage three of Thistlethwaite's algorithm.

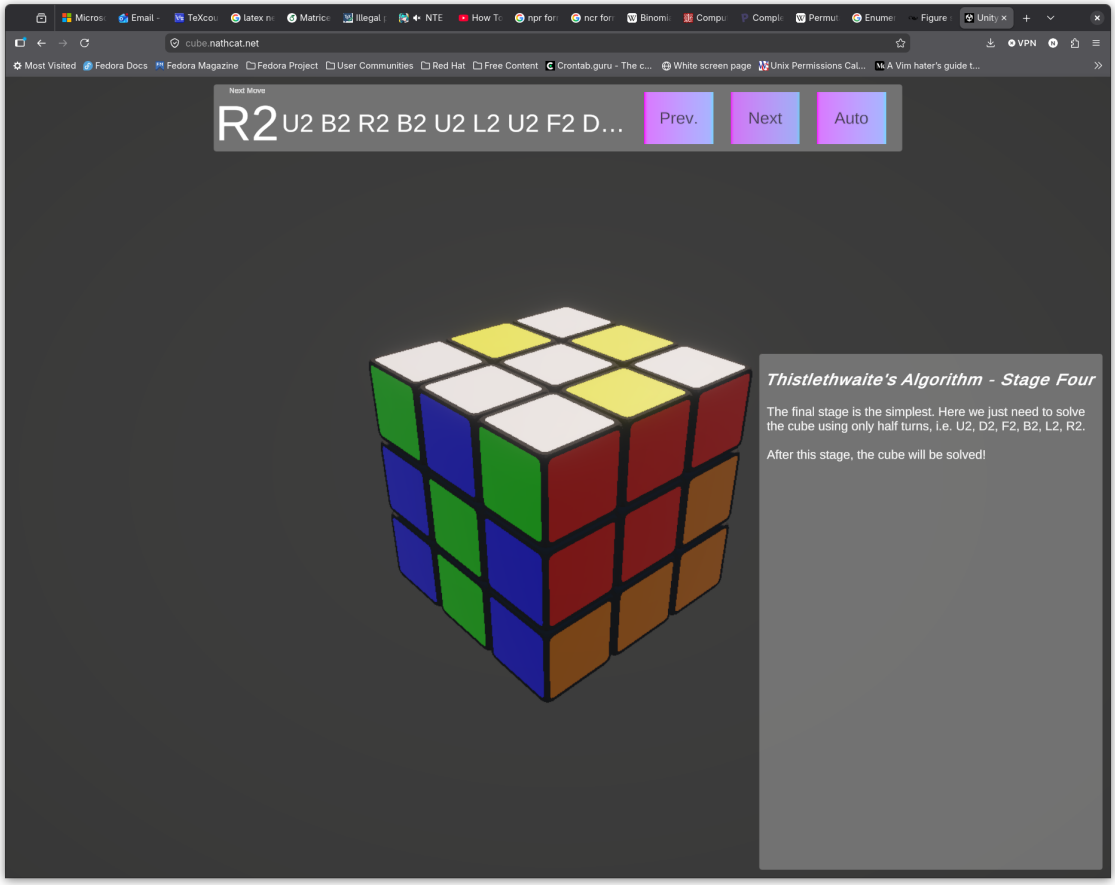


Figure 3.10: Solving the cube, in stage four of Thistlethwaite's algorithm.

Cube painting and state validation

I implementing the cube painting feature using Unity's raycasting feature. When in painting mode, the user can left click to paint a facelet, this action sends out an imaginary ray in the direction they clicked, if this ray collides with a cubie, I can then get the specific position the ray made contact with the cubie, and subtract the cubie's position to get a vector of the hit position relative to the cubie. I then compare this vector to each of the six normal vectors passing through the six facelets of the cubie, and determine which is the closest to our hit position. This then tells us which facelet was hit, and which facelet must be painted.

However, because all six facelets of the cubie can be painted, this means the user will be able to paint the inner facelets, i.e. the facelets which do not face outward on the cube, if they can see them. In order to avoid this, we can ensure that the normal vector of the facelet which was clicked matches the normal vector of any one of the faces the cubie is on.

The primary challenge for implementing state validation is finding ways to determine the properties of the cube with no prior information about the state of the cube. In the cube model I used for generating the solution tables I was able to initialise a solved cube and track the changes to properties of the cube over every turn, such an approach will not be possible here.

As I did with the facelet painting, I can to some extent exploit the geometry of the cube in the three dimensional space to obtain the required information.

Counting the occurrences of each facelet colour is relatively trivial, one must simply look for the painted colours on each cubie. The configuration of the centre facelets is also simple enough, we simply check every opposite pair of centre cubies and ensure that they are valid opposite pairs. We must also ensure that the pairs are correct relative to each other, we can do this by finding the white and green centres, the remaining pair will contain the orange and red centres, the orange centre must be on the left face when the white centre is on the top face, and the green centre is

$$|\vec{v}| = \sqrt{\sum_{i=1}^{n} v_i^2} \tag{3.1}$$

Figure 3.11: Euclidean distance for an n dimensional vector [Smi13]

on the front face.

The next step is to determine whether each cubie is unique or not. This is a slightly more difficult task. We must determine the colours which should make up each cubie. Since the cube will be painted in an arbitrary rotation, we cannot simply say that the top face will be white, front will be green, and so on, we must decide beforehand which face is which (top, bottom, right, etc...), and then get the colour of their centre pieces. With this information we can then determine each cubie. For example, if the front face has the green centre, and the top face has the white centre, the UBL cubie must be the corner which contains the colours white, orange, and blue. We will work through each cubie and attempt to determine which cubie it is using this method, if there are any cubies that appear multiple times, or any cubies which do not match any valid configuration of colours, then the cube state is not valid.

Determining the orientation of each cubie is also a tougher challenge. The orientation of the edges and corners are determined in different ways. I started by defining the normal vector for each face, and the normal vectors for each facelet of a cubie (relative to its own rotation). For each corner then, I took the normal vector for the L/R facelet, and compared it to the face's normal vector, if equal[1], the corner's orientation is zero. Then for each corner I defined vectors for each of the two other orientations, matching the L/R facelet normal to one of these vectors tells us the orientation of the corner. Determining the orientation of the edges was more difficult. After examining the cube, I observed a set of rules which would allow us to determine edge orientation:

- If the edge is on the F or B face, and edge's facelet on the F/B face is an L/R facelet, then it is a bad edge.
- If the edge is on the R/L face, and then edge's facelet on the R/L face is an F/B facelet, then it is a bad edge.
- If the edge is on the U/D face, the edge is on the front or back row of cubies on this face, and the edge's facelet on this face is an F/B facelet, then it is a bad edge.
- If the edge is on the U/D face, the edge is on the right or left row of cubies on this face, and the edge's facelet on this face is an L/R facelet, then it is a bad edge.

If none of these conditions are met, then the edge is good. Assigning an index to each orientation state of a cubie then allows us to 'count' the total orientation of the edges and corners. The cube state is only valid if the total edge orientation is even, and the total corner orientation is divisible by three.

Finally, we can use Pochmanns parity method (See Figure 3.4) to determine the overall parity of all cubies. If the cube state is valid, the result must be even.

Description of each stage

The solution is split up into four stages, when each stage is being executed on the display cube, a brief description of the goals of the stage is displayed.

Key points of the solve are also highlighted:

- Stage one
  - Good edges highlighted in green
  - Bad edges highlighted in red

[1]To account for floating point errors, I actually made it test whether the difference between the vectors (defined by their euclidean distance (See Figure 3.11)) was within some small margin.

- Stage two
  - Correctly oriented corners are highlighted in green, and red if incorrectly oriented.
  - M slice edges are highlighted green when in the M slice, and red otherwise.
- Stage three
  - Each corner pair is highlighted in a different colour.

I have not highlighted anything in stage four since there isn't really any key points to highlight, we are simply solving the cube with only half turns.

3.3.2 Timer and Speedcubing Database

Because of the fact that I was unable to start my implementation earlier, and that my implementation of Thistlethwaite's algorithm was *significantly* more difficult and time consuming than I had anticipated, I was unable to find the time to implement these parts of my project.

Following feedback from the marking of my interim report, it appeared that these the implementation of these parts of my project were not as vital as my implementation of Thistlethwaite's algorithm and the solver application.

3.4 Evaluation

Since I was not able to complete the timer, and algorithm database, let us examine the solver specification.

- Implementation of Thistlethwaite's Algorithm & Lookup table generator.

This formed the largest part of my project in the end, and I was able to implement it successfully. The only change I made to Thistlethwaite's original specification was using Pochmanns variant of the third stage [Poc].

In fact my implementation of the algorithm actually produces much better solutions than I had originally anticipated. Thistlethwaite states in his letter describing the algorithm that he would expect somewhere between 45 and 52 moves for solutions [Sch15b], over 100000 random test solves, my implementation averaged 20 moves, a figure not too far off Kociemba's God's number [Koc+13], the lower bound of moves to solve any arbitrary position of the cube.

- Server which accepts cube permutations and returns a sequence of moves to solve said permutation.

This part of the implementation was also successful. The solver site is able to accept a cube permutation encoding, extract a cube model from this encoding, and determine a solution using Thistlethwaite's algorithm, serving this algorithm back to the client. Since the algorithm is simply four sequential lookups and then a concatenation of the solutions, the solutions are also served very quickly.

- Allow users to paint a cube to the state they wish to solve

My implementation of the solver site allows the user to paint a cube state, they are presented with a blank cube, and six coloured buttons. They may click the coloured buttons to change the colour they are painting, and then click on the cube's facelets to paint them in their selected colour.

- Site determines permutation and sends to the server to solve

The solver site is able to obtain all the necessary information from the cube's geometry within the three-dimensional space to produce a permutation encoding as required by the solving server (See Figure 3.5).

It is also able to make an HTTP request to the server, which returns the solution.

- Site animates the solution step by step

Once the site has received the solution from the server, it displays it at the top of the screen in Singmaster notation [Joy08], with three control buttons. One button reverses the previous move, one performs the next move, and the other automatically plays through the solution, see Figure 3.7.

Each turn of the cube is animated.

- Information about the algorithm used to solve the cube and each stage should be displayed.

A brief description of each stage of the algorithm is displayed on the right side of the solver view with the solution for that stage. Key points of the stage are also highlighted. In the first stage for example, good edges are highlighted in green, and bad edges in red.

The description provided is not in-depth, but just briefly describes the goals of each stage.

Chapter 4

Conclusion

4.1 Success of the Implementation

While I did not manage to implement everything I set out to, what I did manage to achieve was, I think, very successful. I have sufficiently met the requirements I set out for the solver application, and in some parts of that even surpassed my own expectations. The average move solution length of my implementation of Thistlethwaite's algorithm for example, is significantly lower than what I would have expected, and is surprisingly close to Kociemba's God's Number [Koc+13].

My reason for not having managed the remainder of the project is, I think, simply that I underestimated the complexity and sparsity of documentation of Thistlethwaite's algorithm. The same was the case with my interim report, my description of Thistlethwaite's algorithm was then very incomplete, but after having done more research into the topic, and exploring the algorithm through my own implementation, I have developed a much deeper understanding of it. Scheduling issues also played a part, I had originally planned on starting the development of this project over the Christmas break, but was unable to find the time to do so between everything I was dealing with at the time.

Regardless, I do believe my implementation to have been a success, and I am more than happy with what I have achieved here.

4.2 The Bliss of Hindsight

In my original plan, I had allocated 1-2 weeks for the implemnentation of Thistlethwaite's algorithm, but as a result of the relative lack of resources on the topic, and the fact that a lot of those resources were very difficult to decipher, my implementation ended up taking to better part of two months to complete. My efforts in this were also hindered by my choice of language, nevertheless, this project has served as a valuable learning opportunity about writing memory intensive programs in Java vs. C++.

I would've liked to begin my implemnentation earlier, although as I mentioned before I simply could not find the time, and I have undoubtedly suffered the consequences of this.

Despite the difficulties that this project has posed, I have learned a lot from it.

In the end I think I have achieved a different goal. While I set out to create a hub for speedcubers to practice and learn new skills, I have ended up learning a lot about Thistlethwaite's algorithm, and I'd like to think I have helped create a more cohesive documentation of it, and its implementation. In my research I found it very difficult to find any reasonable source or documentation of the algorithm and its theoretical background, so I hope that my efforts in this report might help someone else with this problem in the future.

4.3 Future Extensions

In the future, I would like continue with this project, working to implement the parts of proposed application that I was unable to implement here. I would also like to deploy this on my own server

setup so that it is publically accessible.

However I think the biggest thing that I would like to implement is more solving algorithms. The main goal of this project was to create a centralised place where speedcubers could learn about techniques and new skills, so implementing some human solving algorithms, like CFOP [Fri97], into my solver would be very useful to this task.

I would also like to think about extending my solver to work with $N \times N$ size cubes. Initially in my development I tried to with this goal in mind, but it ended up being too complex for the scope I had already laid out. It would require a lot of work and probably a complete rewrite of my solver application under another framework which can more easily support arbitrarily sized cubes. An adaptation of Thistlethwaite's algorithm for the purpose of solving cubes like this would also be very interesting. The human methods behind solving $N \times N$ cubes normally rely on reduction, i.e. they solve the cube into a state that can be interpreted as a standard 3×3 cube, and must then solve from there using an existing 3×3 solving algorithm. Decomposing this reduction problem into stages as Thistlethwaite has done with his 52-move algorithm would be an interesting challenge.

Bibliography

[BW] Alan Bundy and Lincoln Wallen. "Breadth-First Search". In: *Catalogue of Artificial Intelligence Tools*. Ed. by Alan Bundyand Lincoln Wallen. Berlin, Heidelberg: Springer Berlin Heidelberg, 1984, pp. 13–13. ISBN: 978-3-642-96868-6. DOI: 10.1007/978-3-642-96868-6_25. URL: https://doi.org/10.1007/978-3-642-96868-6_25.

[Goe87] P. Goetgheluck. "Computing Binomial Coefficients". In: *The American Mathematical Monthly* 94.4 (Apr. 1987), pp. 360–365. URL: https://www.jstor.org/stable/2323099.

[Fri97] Jessica Fridrich. *The Fridrich Method (CFOP)*. 1997. URL: https://www.cubefreak.net/speed/cfop/index.php.

[Kor97] Richard E. Korf, ed. *Finding Optimal Solutions to Rubik's Cube using Pattern Databases*. AAAI-97. 1997.

[Cha02] C.A. Charalambides. "Enumerative Combinatorics". In: Discrete Mathematics and Its Applications. Taylor & Francis, 2002, p. 43. ISBN: 9781584882909. URL: https://books.google.co.uk/books?id=PDMGA-v5G54C.

[Dav06] Tom Davis. *Group Theory via Rubik's Cube*. 2006. URL: http://www.geometer.org/rubik/group.pdf.

[Gro08] Jonathon L. Gross. *Combinatorial Methods with Computer Applications*. Taylor & Francis Group, 2008.

[Joy08] David Joyner. "Elements, my Dear Watson". In: *Adventures in group theory : Rubik's cube, Merlin's machine, and other mathematical toys*. Baltimore, Johns Hopkins University Press, 2008, p. 26.

[Koc+13] Herbet Kociemba et al. "The Diameter of the Rubik's Cube Group is Twenty". In: *Siam J. Discrete Math* 27.2 (2013).

[Smi13] Karl Smith. *Precalculus: A Functional Approach to Graphing and Problem Solving*. p. 8. Jones & Bartlett Publishers, 2013.

[Sch15a] Jaap Scherphuis. *Kociemba's algorithm*. 2015. URL: https://www.jaapsch.net/puzzles/compcube.htm#kocal.

[Sch15b] Jaap Scherphuis. *Thistlethwaite's 52-move algorithm*. 2015. URL: https://www.jaapsch.net/puzzles/thistle.htm.

[WCA25a] World Cubing Association (WCA). *Regulations, Article 4b3 — Scrambles*. 2025. URL: https://www.worldcubeassociation.org/regulations/#4b3.

[WCA25b] World Cubing Association (WCA). *Regulations, Article 9b — Events*. 2025. URL: https://www.worldcubeassociation.org/regulations/#9b.

[Edi25] Britannica Editors. *Erno Rubik*. 2025. URL: www.britannica.com/biography/Erno-Rubik.

[NN] Katherine Nogin and Maria Nogin. "Permutations in Rubik's Cubes". In: *Math Horizons* 32.3 (2025). URL: https://doi.org/10.1080/10724117.2025.2450987.

[NIS] NIST. *NIST Digital Library of Mathematical Functions*. https://dlmf.nist.gov/, Release 1.2.4 of 2025-03-15. F. W. J. Olver, A. B. Olde Daalhuis, D. W. Lozier, B. I. Schneider, R. F. Boisvert, C. W. Clark, B. R. Miller, B. V. Saunders, H. S. Cohl, and M. A. McClain, eds. URL: https://dlmf.nist.gov/.

[Poc] Stefan Pochmann. *3x3 Thistlethwaite Solver*. URL: https://www.stefan-pochmann.info/spocc/other_stuff/tools/solver_thistlethwaite/solver_thistlethwaite_cpp.txt.

Appendix A

Interim Log

In the following section you will find logs of the biweekly meetings I had with my supervisor, Dr. Alessio Santamaria.

A.1 Meeting 1 — 2nd October 2025, 4pm

Discussed the initial steps to be completed, and what the timeline over the year duration of the project would look like. Focused a lot on the content of the interim report and the initial project proposal.

Notably the aim of the interim report is to present a detail plan with all technical details of the project. It is not expected that anything has actually been produced yet besides this report, but the idea is that anyone should be able to read this report, and understand the work that needs to be done to create the product.

Also briefly discussed the potential use of my own authentication system *AuthCat*, as a backend authentication system, and some of the implications using it would place on the content of my project. Namely the fact that I will need to provide technical specifications for that system, proving that I built it myself and fully understand what I built.

While it will take me some extra time to create these specifications, this should not be a problem.

A.2 Meeting 2 — 16th October 2025, 4pm

Discussed improvements to my project proposal in anticipation work commencing on my interim report:

- Make use of URL macro for URLs.
- Find a better reference for Thistlethwaite's algorithm.
- The use of AuthCat.

The main discussion point during this meeting being my use of AuthCat in the project. Following his inquiry, I would be able to use it, given that I have made it myself, but it would not be included in the grading I would receive for my project, since it was not implemented as part of the project. Furthermore, using it would create more ethics implications, since it is used in an array of applications, I would have to prove not only that the system is secure, but that all of the applications that use it are ethical.

While this is of course possible, I think the best course of action for my project would be to implement a new authentication system for this project, but in order to support further deployment / maintenance after the project is complete, I could develop it in such a way that the authentication system in the back end is modular, and could be swapped out for another one which follows the same specification. This effectively creates an *authentication plugin* structure, so for this project I

can use a new system, and if I end up deploying this application into use later, I can write a new authentication module which integrates with AuthCat.

A.3 Meeting 3 — 30th October 2025, 4pm

Discussed the implications of using `https://www.jaapsch.net/` as a citation for Thistlethwaite's algorithm. Since this is the best source I can find for that algorithm I can use this as a citation, but I will add a footnote clarifying that this is the best I could find.

Also discussed the contents of the Professional Considerations section, i.e. I simply need to go over the BCS Code of Conduct and relate its points back to my project. We also talked about how best to clarify that I will not be conducting user testing, and instead will be comparing my final project against a more explicit set of criteria which I lay out in the requirements analysis section of this report.

A.4 Meeting 4 — 6th November 2025, 4pm

This meeting was moved forward a week, since two weeks from the previous meeting would have been right after the deadline for the interim report.

Looked over my Professional Considerations section, and a brief look over the rest of what I have written at this point. Dr. Santamaria's main suggestions were that I should include a theoretical introduction to the Rubik's Cube before getting into the more technical concepts, and that I should include a more explicit design of how the final web application will look, meaning the pages which are available, what they might look like, etc....

A.5 Meeting 5 — 4th December 2025, 4pm

Covered feedback from the marking of my interim report. I felt the following were the key take-aways:

- Consider the implementation of a real time cube state validation algorithm that works on the cube's facelets.
- The introduction to the theoretical side should be clearer, and some parts are not explained. In particular the figure which uses Disjoint cycle notation, has no background on what disjoint cycle notation is.
- Are the listed conditions sufficient for a cube state validation algorithm? How can I prove this? Is there some academic source which might provide a formal definition of such an algorithm, rather than just making use of my own intuition from background research?
- The background research and requirements analysis are generally interleaved, which leaves it relatively unclear. Perhaps these sections should be separated.
- The requirements analysis itself is presented in an essay style, and as such does not necessarily present a clear picture of what I am trying to achieve.
- There is not much discussion of the target end user's in the requirement's analysis.
- Grammar mistakes, which are apparently common for my age group. The sentence 'A transposition being defined as a cycle of two elements.', is an example. In fact I think this point may be an example. I should look into this particular grammatical error and rectify it.

A.6 Meeting 6 — 3rd February 2026, 10:30am

Went over some specifics of the final written report. In particular the required depth of the background research section, and the level of detail required in the implementation / technical analysis.

- It is required to cover what is needed to understand the technical aspects of the project, thus it may not need to be as in depth as the interim report.
- Analysis of the technical implementation is intended to be at a relatively high level. It should explore the details of the implementation, but does not exactly need to walk through the code line-by-line. Such questions might be asked in the presentation instead, but questions answered by the discussion of implementation in the final report will not be asked.

A.7 Meeting 7 — 17th February 2026, 10:30am

More or less just a progress update in the development of my project. Discussed the implementation of the lookup tables, the BFS search algorithm used to generate them, and my current challenges with the corner orientations.

A.8 Meeting 8 — 17th March 2026, 10:30am

Very similar to the previous meeting. Thistlethwaite's algorithm posed an incredible challenge to implement.

At the time of this meeting I am one stage away from completing the algorithm, and had some good ideas on how to proceed with it.

Dr. Santamaria reminded me that he would read a draft of my final report and guarantees feedback in a reasonable time frame should he receive said draft at least three weeks before the deadline.

We also briefly discussed some aspects of the final report structure, in particular what is expected of the statement of originality.

A.9 Meeting 9 — 21st April 2026, 10:30am

Dr. Santamaria shared some initial opinions on my first draft of this report, main impression was that my use of two text columns was unnecessary.

We also discussed the issues I had with my references from my interim report, so that I might be able to fix them in my final report.

Finally, we discussed the expectations for the presentation:

- 15 minutes of presentation and live demonstration.
- 5 minutes of questions
- Should have slides prepared along with some talking points.
- Demonstration doesn't need to show off everything, just the key points of the project.

Appendix B

Project Proposal

B.1 Aims and Objectives

B.1.1 Aims

Speedcubing is defined as the activity of solving a Rubik's Cube or related puzzle as quickly as possible.

Speedcubing has evolved over time since the invention of the Rubik's cube in 1974, gaining popularity and attention. Many tools exist to help cubers learn and develop their techniques, from timers such as *cstimer.net*, and algorithm databases like *speedcubedb.com*. A lot of these tools however, lack certain features, and the sheer number and variation of them can make then hard to find or navigate.

The aim of my project is the take inspiration from these existing tools, and collate their essence into a single hub for Speedcubers, containing all the tools they might need to help them learn and develop, and introducing further quality of life improvements.

I myself have been a Speedcuber for about six years, and have had experience with a lot of different community-led tools. I plan to draw on the experiences I have had with them to produce something which could genuinely help other Speedcubers.

B.1.2 Objectives

There will be three main parts to my product:

- A timer
- A solver
- An open source database for speedcubing algorithms

In this section I will go into some detail about what each of these parts contain, and their purpose within the project.

Cube Timer

The cube timer, refers to an application which both provides Speedcubers with competition grade scrambles, and allows them to time their attempt at solving said scrambles.

A *Competition grade scramble* refers to a scramble sequence generated by a program which meets the *World Cubing Association's (WCA)* specifications for a scramble program, i.e. a random selection from all puzzle states which require at least two moves to solve, with additional requirements for some puzzles, such that each state has an equal probability of being selected [WCA25a].

The timer itself should also provide some statistics, those which are most commonly assessed in WCA competition events [WCA25b], such as:

- Average time of five solves

- Mean time of three solves

Although not stated in the WCA regulations as an official event for a competition, a best single solve statistic can also be helpful.

The key features of this part of the program then form the following list:

- A timer interface
- A means of collecting and displaying the aforementioned statistics
- An interface which provides competition grade scrambles

The final of these features implying the implementation of a scramble program. A commonly used algorithm for scrambling the Rubik's Cube is Kociemba's algorithm [Sch15a]. Kociemba's algorithm is actually a solving algorithm based off Thistlethwaite's 52-move algorithm [Sch15b], but it can also be applied to generating a scramble, effectively by generating a valid random state of the cube, and applying Kociemba's algorithm to check that this state meets the minimum move count requirements.

A further requirement of this part of the application would be that it stores a user's session data in some external database, linked to their account. This would allow them to have persistent sessions across any device they may wish to practice their speedcubing on. This is a particularly important feature to me, because a lot of the existing cube timer applications do not offer something like this, and I always find myself wishing they did.

The account system backing this data storage might be outsourced to an existing project of mine, *AuthCat*. AuthCat is a Single-Sign-On service I created in October 2024 for the purpose of having a consistent user account system across all my applications. I could make use of its account system to allow users to login / create securely stored credentials, and link their persistent session / solve data to them.

This account system will also prove useful in the other parts of this application.

Solver

This part of the application would present users with a 3-dimensional rendering of a Rubik's Cube, allow them to enter a state for the cube, and then display the sequence of moves that would solve the state they entered using animations, and Speedcubing notation [Joy08].

The scrambler in the cube timer section of the application would require the implementation of a cube solver algorithm, so the primary challenge in this part of the application would be the 3D rendering and animations.

A stretch goal in this section might be to include an option to solve the cube either using Kociemba's algorithm, or a Speedcubing method such as *CFOP*, illustrating each stage of the solve as it goes. This would be a very helpful feature to implement for Speedcubers trying to learn new solving mechanics, but is not necessary to the base function of this section.

Speedcubing Algorithm Database

The aim of this part of the application would be to display common speedcubing algorithms, such as the set of 21 algorithms used in the *Permute-Last-Layer* stage of the CFOP method, and allow users to post move sequences which solve each of the cases the algorithm deals with, and also provide a way for them to vote for their favorites.

This might allow speedcubers trying to learn these algorithms to find the move sequence that fits their style the most. I can personally attest to how useful something like this would've been to me when I was learning the CFOP algorithms, there are lots of sites out there which detail move sequences for each of the cases in the CFOP method, since it is one of the most popular speedcubing methods, but they are generally scattered around, and it can be difficult to find the algorithm that you like the most. This process would be much easier if they could all be collected into one place.

B.2 Relevance

This project, while relatively simple on the face of it, contains within it a lot of areas which have featured in my degree, to name a few:

- Web development — This will test skills learned in Multimedia based modules, including Introduction to Multimedia, and Human Computer Interaction
- Database development and management — This will test skills learned in Databses, and Data Structures and Algortihms
- Source control
- Academic reading / writing
- Implementation of algorithms detailed in an academic paper or specification — This will test skills learned over the course of my entire degree, with the implementation of Kociemba's algorithm in particular testing skills learned from modules like Data Structures and Algoriths, and Mathematics for Computer Science.
- Cybersecurity

These are also some of the areas I find myself most interested in pursuing in my later career, whether that end up being in research, or software development.

While my practical programming skills have been strong for many years, the elements of this project which rely on the understanding of theoretical elements I think will test the skills that I have learned over the course of my degree at Sussex.

B.3 Resources Required

- Front and back end web hosting (Most likely a combination of Apache and Java. I would most likely use Amazon Web Services to provide the hardware resources for this)
- A database system (MySQL)

B.4 Weekly timetable

Time	**Monday**	**Tuesday**	**Wednesday**	**Thursday**	**Friday**
09:00	Lab		Lec.		
10:00	Lab	Project			Project
11:00		Project	Project	Lec.	Project
12:00		Project	Project	Lec.	Project
13:00		Project	Project		Project
14:00		Project	Project		Project
15:00		Project	Project	Lec.	Lec.
16:00	Lec.	Project	Project		Project
17:00	Lec.	Project	Project	Lab	Project
18:00			Project		Project
19:00	Lec.				Project
20:00					

Appendix C

UI Design

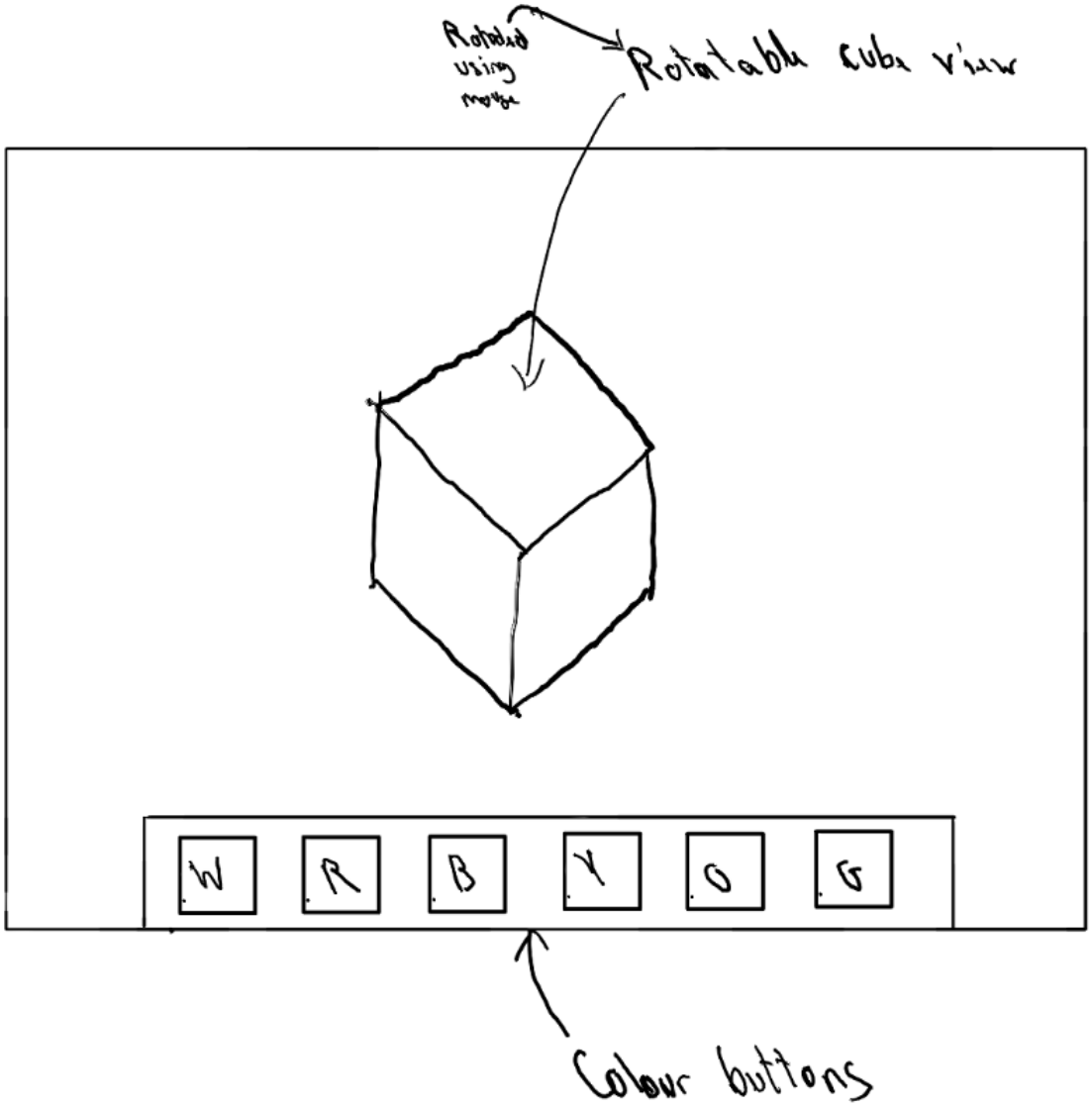


Figure C.1: The page you are greeted with when opening the solver. This page will allow you to enter a cube state.

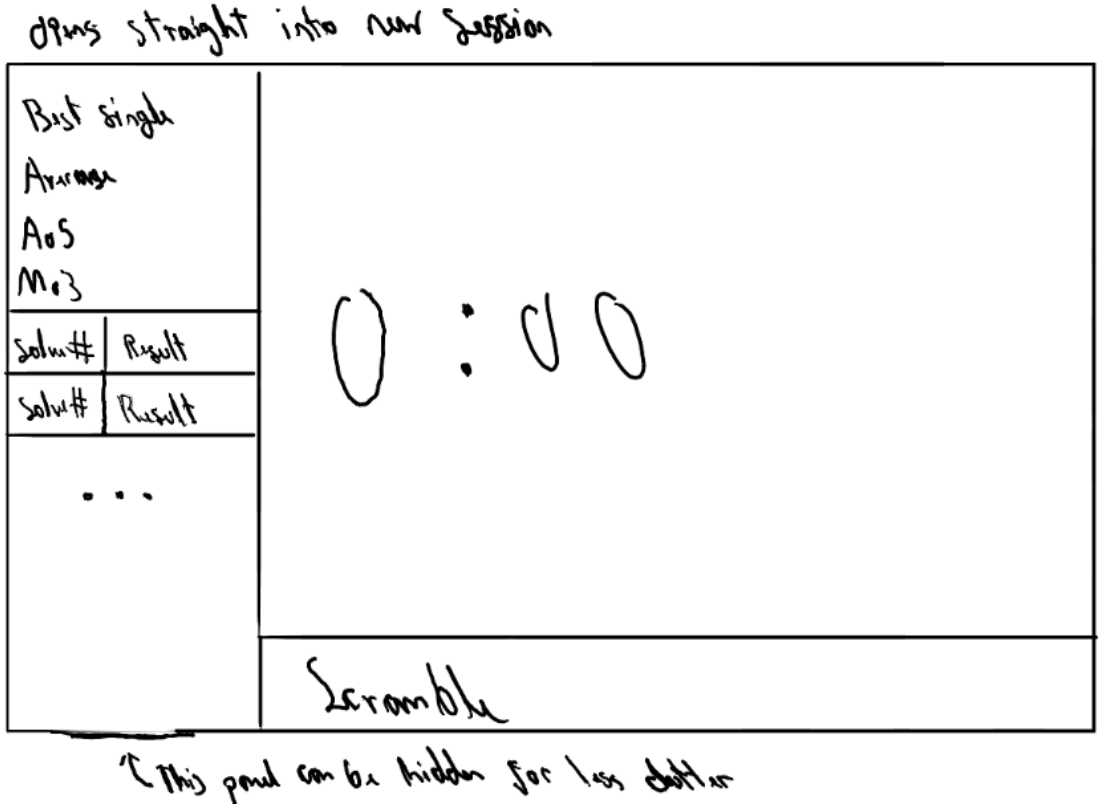


Figure C.2: The main page of the timer. Upon opening the timer, the timer will create a new session. The statistics panel can be hidden to reduce clutter.

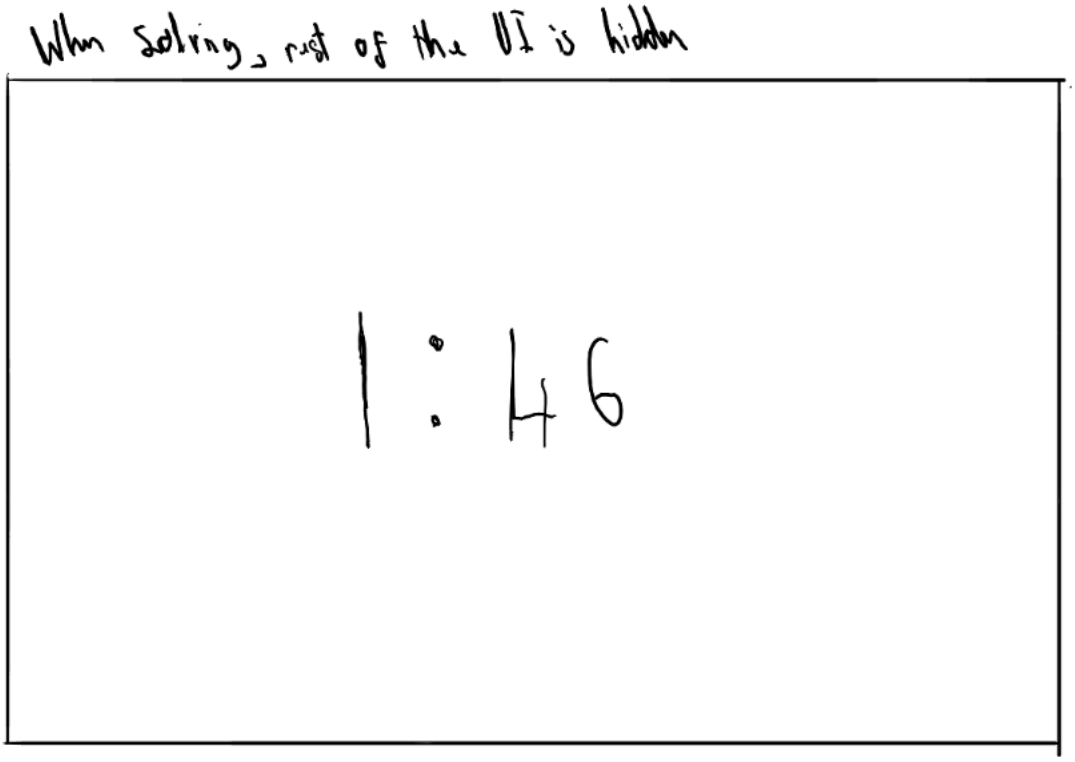


Figure C.3: When solving, most of the timer UI will be hidden to reduce clutter and distraction on the screen.

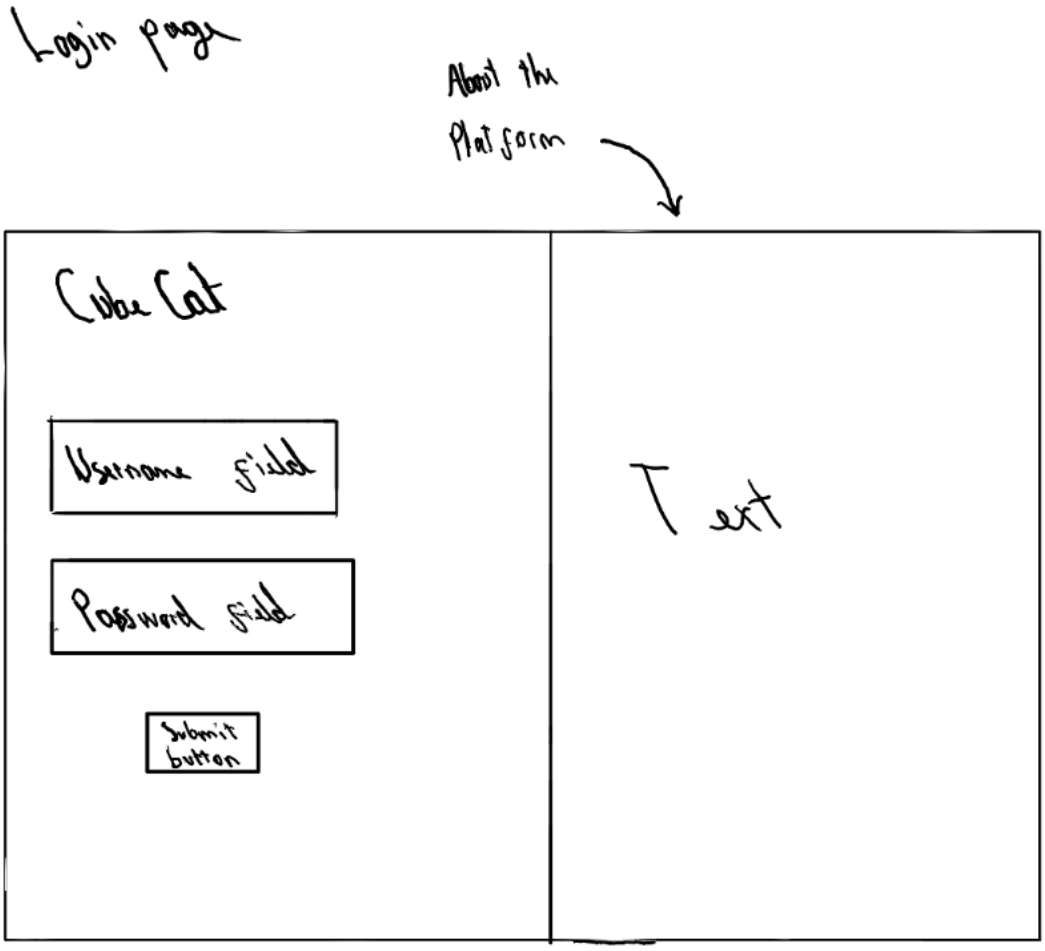


Figure C.4: The login page. Every page pf the application will redirect to here if the client is not already authenticated.

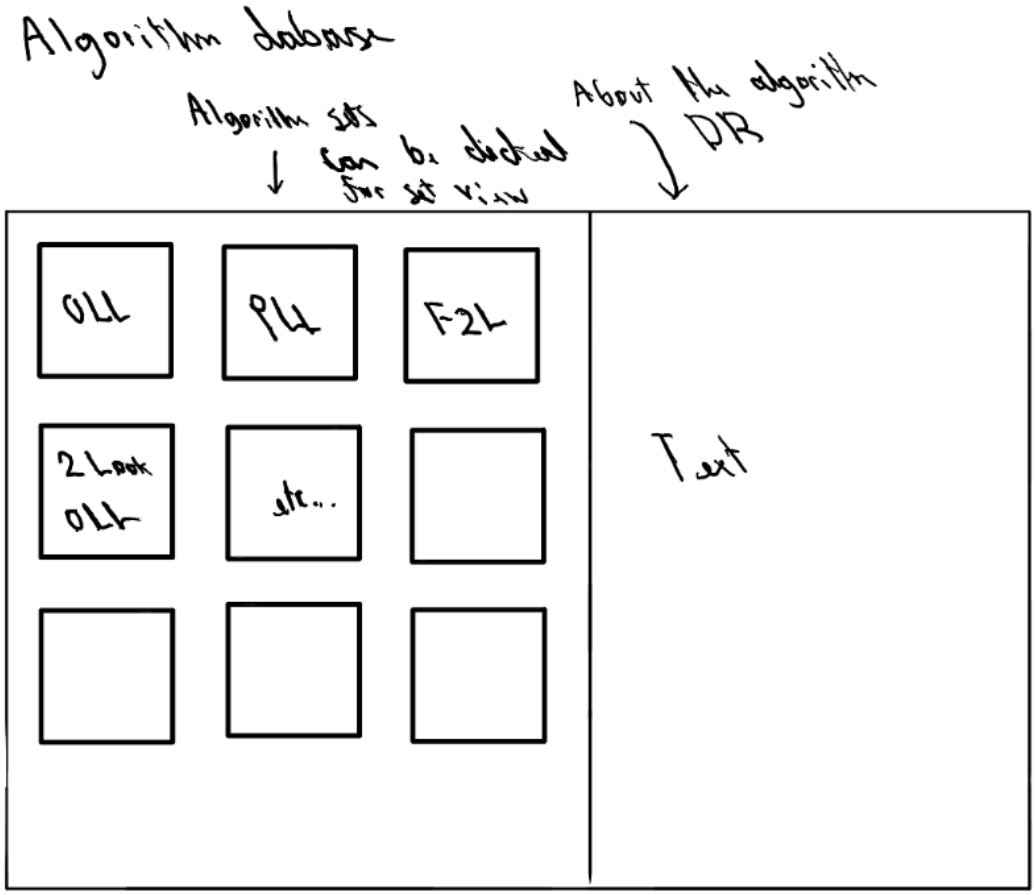


Figure C.5: The main page of the database. This shows all algorithm sets that are available on the database. Each set may be clicked to got to the set's page.

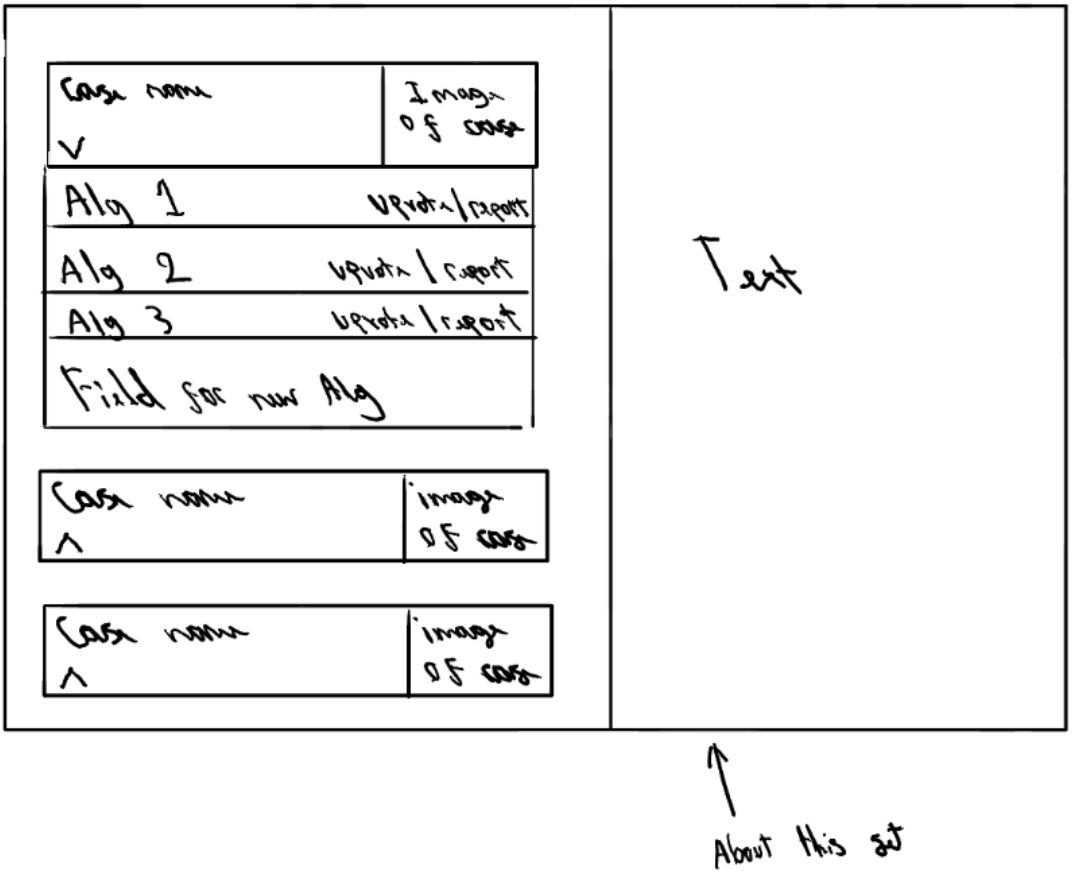


Figure C.6: This page shows the cases in an algorithm set, with dropdowns for each case to show algorithms that the community has submitted. The bottom row of this dropdown list will be a text field which allows the user to submit a new algorithm.

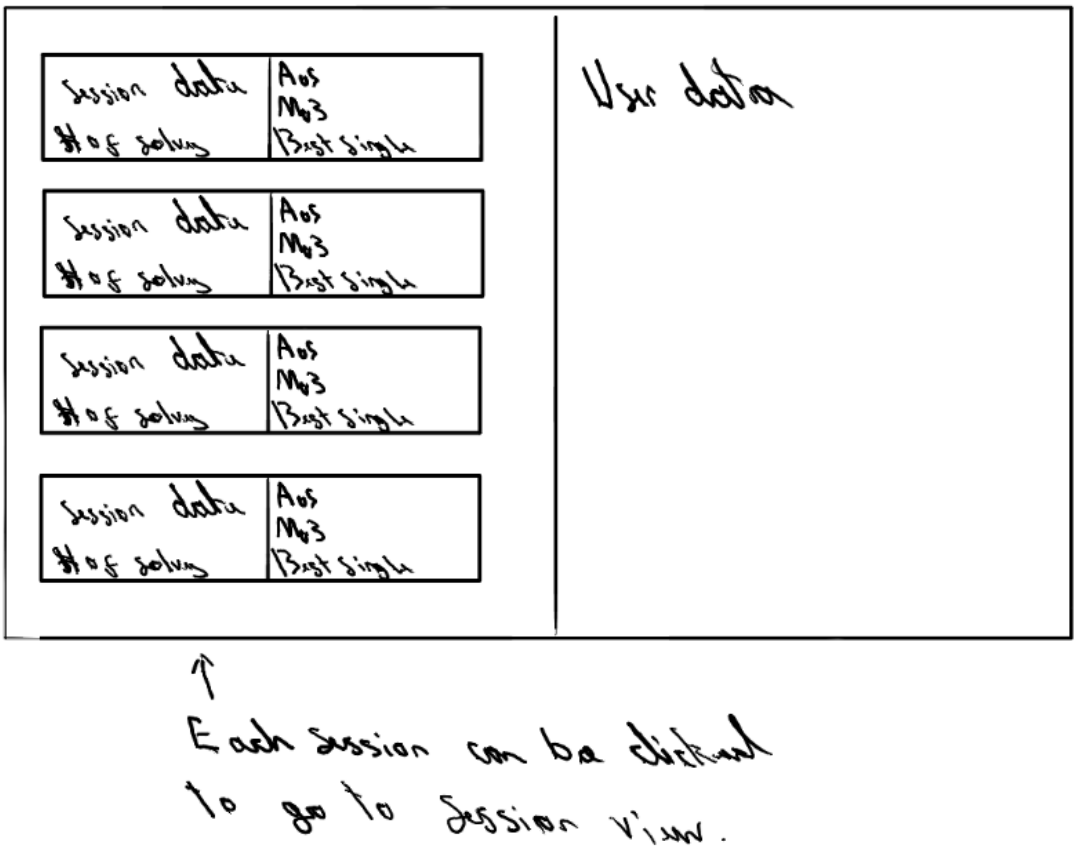


Figure C.7: This page shows all of the user's completed sessions.

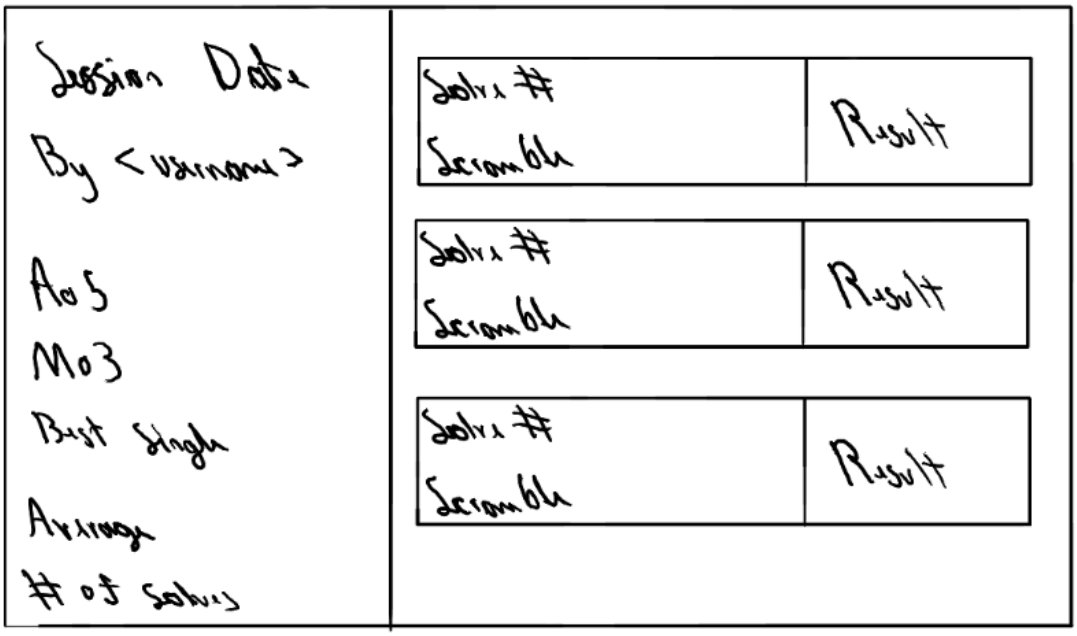


Figure C.8: This page shows in more detail one particular session completed by a user. This page does not require authentication so can be viewed and shared by anyone.

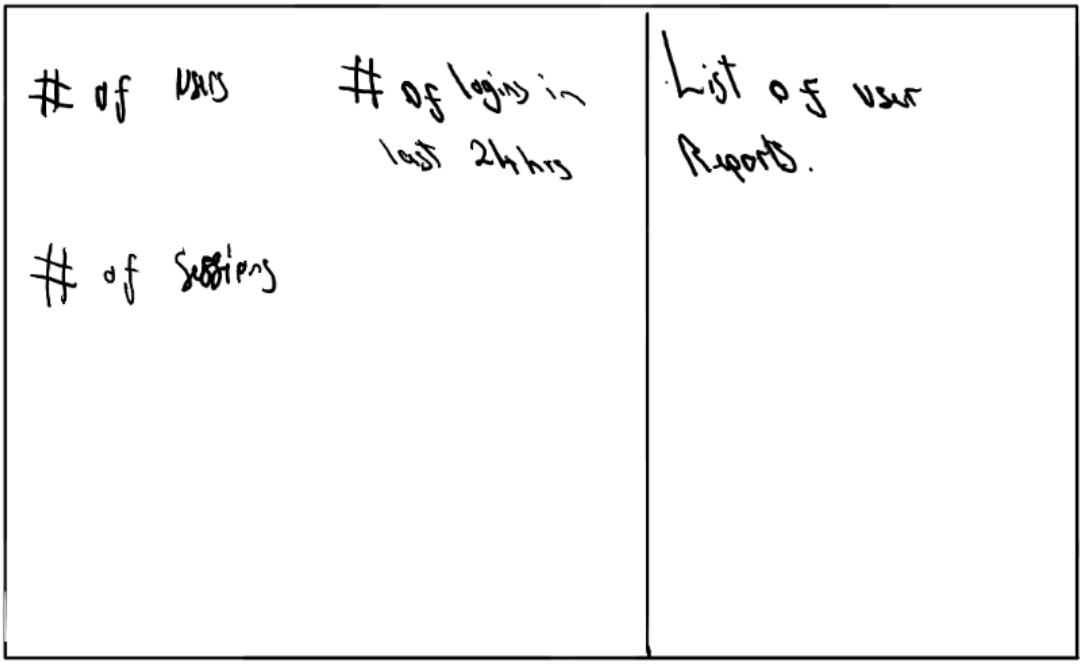


Figure C.9: This is the admin page. Showing statistics about the platform, and a list of reports made by users.